Chapter 1

Introduction to Neural Computation

Questions !

Why computers which have high computing abilities, but still Cannot do simple things the human brain can do easily …

For example, Face Recognition, Listen and Understand Speeches

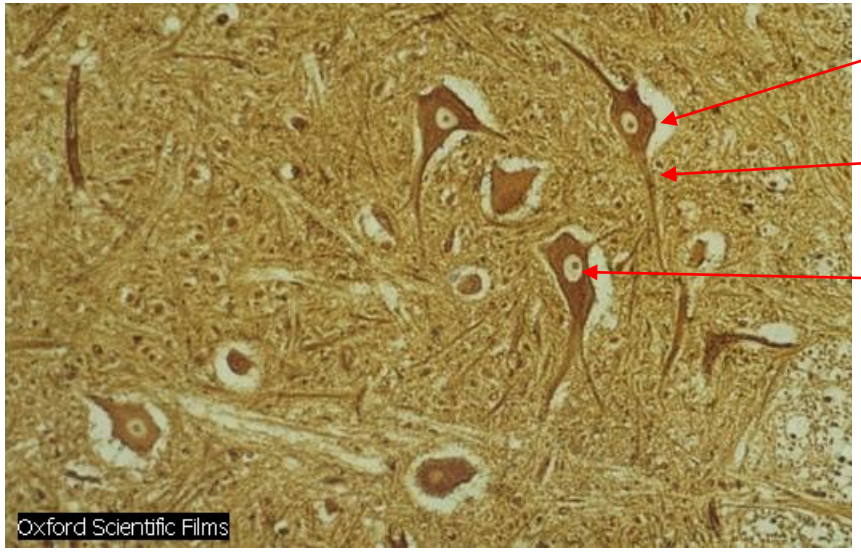
Human Brain vs Computer

Speed	Neural Cells send Signals at rate 1000 times/s	Speed of Computer Now at > 3000 MHz
Math Computing	Brain 👎	Super Computer > 1000 MFLOP
Memorize and Classify Pictures and Sounds	Brain 👍	Computer 👎

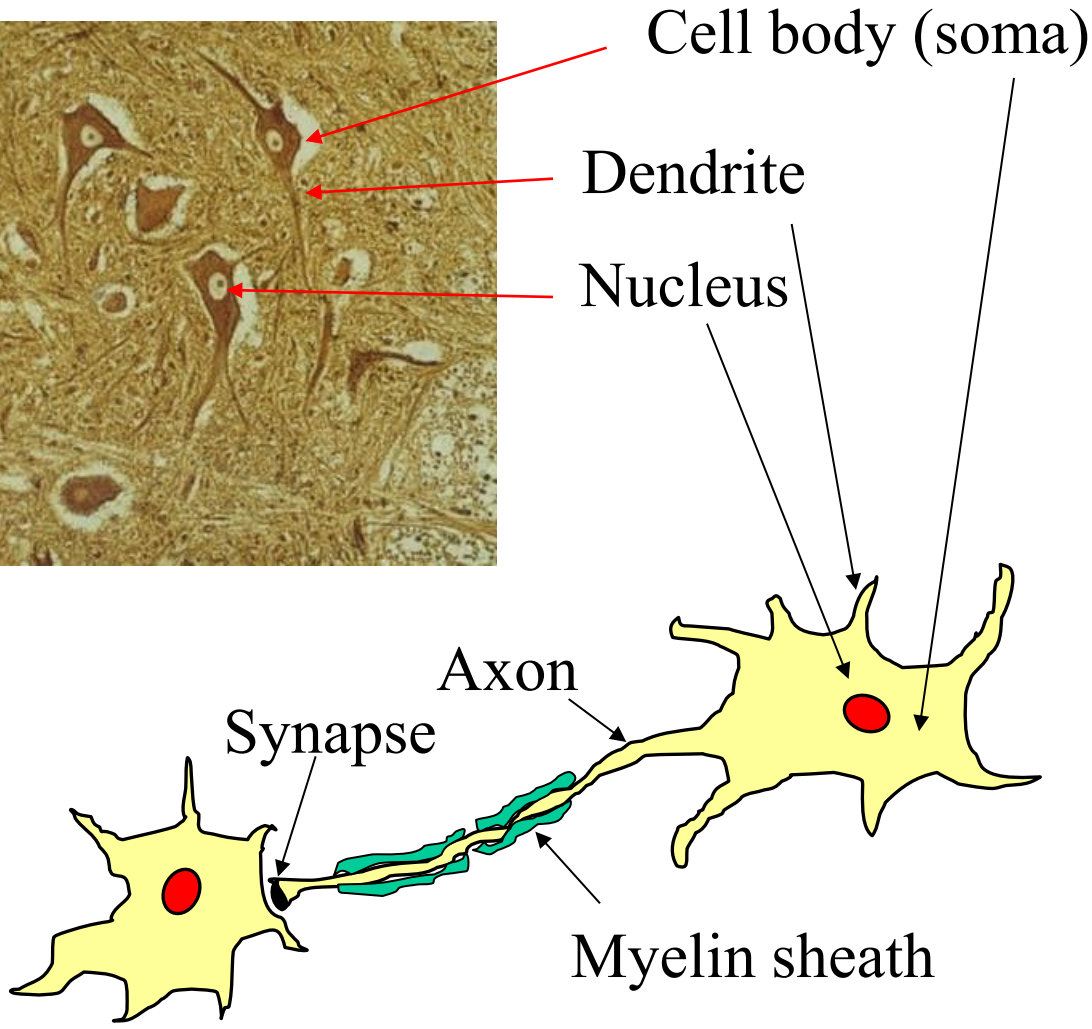
Human Brain

- ✓ Human Brain is powerful and stable
  (every day neural cells are dead)
- ✓ Human Brain is flexible (learning)
- ✓ Human Brain capable of handling data with uncertainty, noisy, unstable
- ✓ Human Brain computes huge data with parallel process
- ✓Human Brain is small and require little power
- ✓Human Brain structure is evolved more than million years
- ✓And still survive now

Structure of Brain and Neural Cells

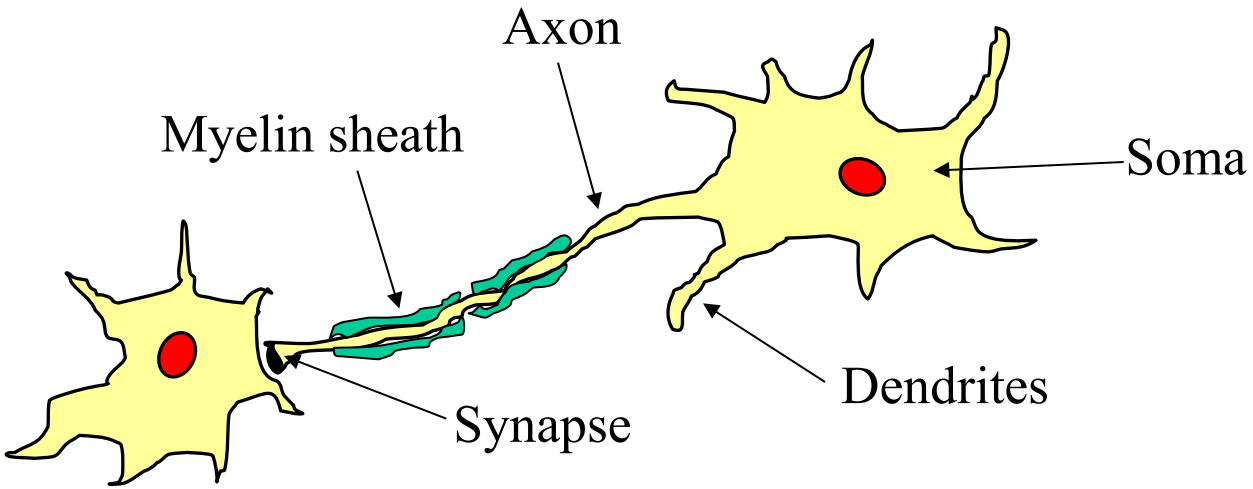


Neuron



How a cell works

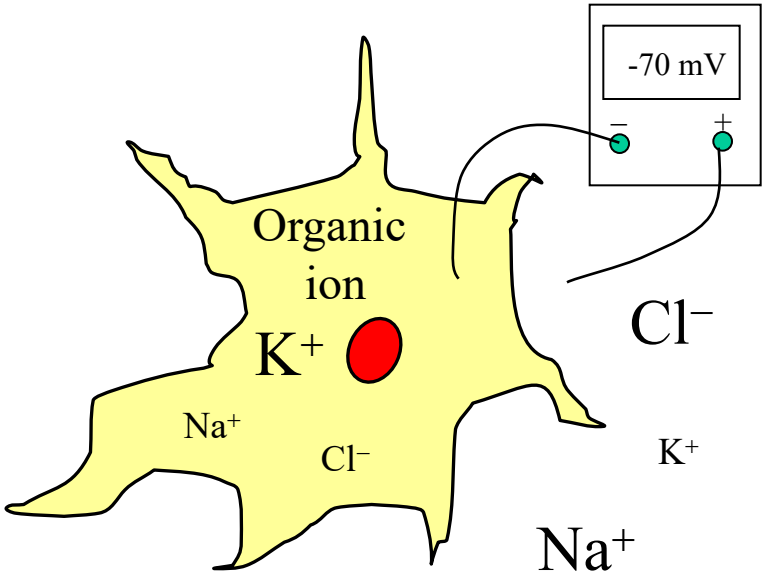
- Neural Cells consist of Soma, Axon and Dendrites
- at the end of Axon there is a Synapse connected with Dendrites of other cells, in which Axon and Dendrites are used to send electrical signal between cells
- Axon is longer than Dendrites (Axon may be long for 1 meter)
- Axon has Myelin sheath for getting better signals and remove noises



How a cell works

-Normally when Neuron does not work, inside the cell will have high $K^+$and Organic ion, and outside will have high Na^+ and Cl^-

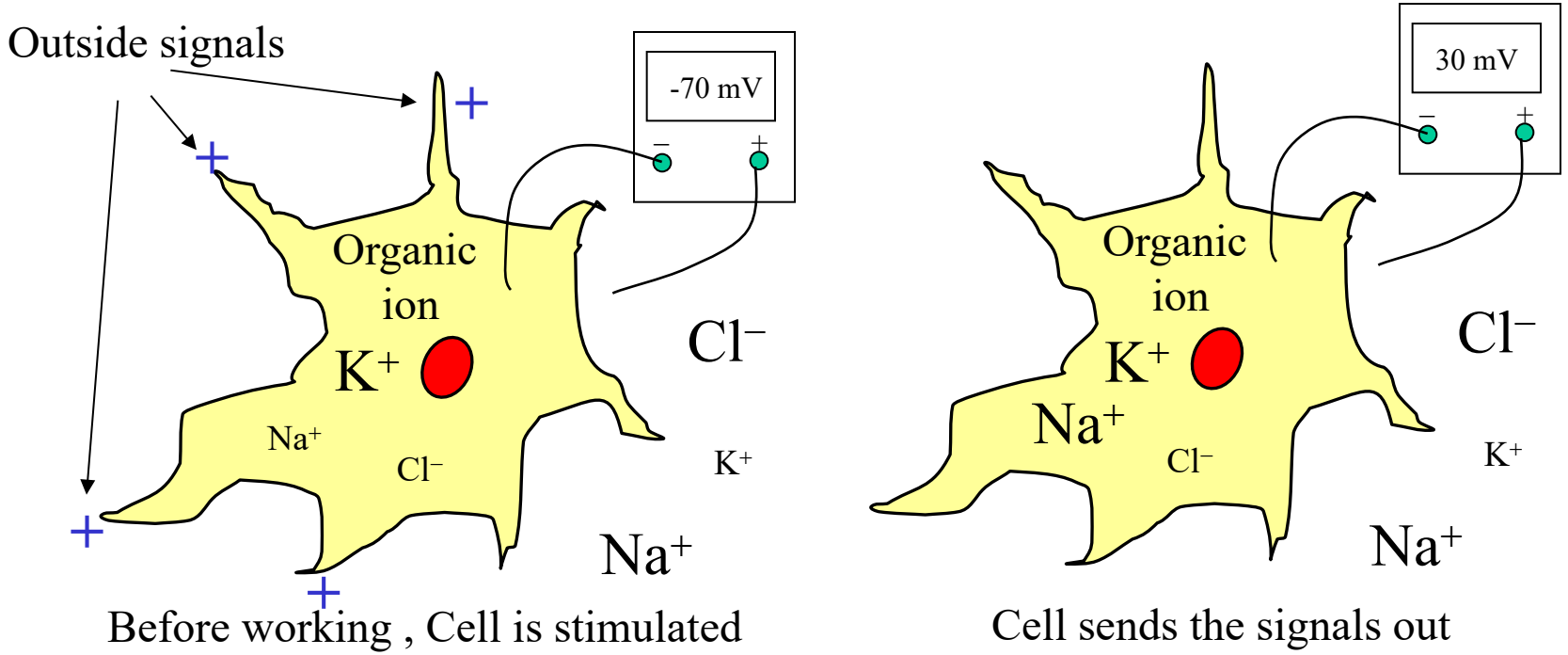
- When it is stable, the different between inside and outside the cell makes voltage about -70 to –100 mV called *Resting potential*



How a cell works

- Electrical signals are sent via Axon and Dendrites to Soma

- If there is enough signal at Soma, the concentrate of ion inside will increase because of increasing of Na+ then making the Resting potential decreases,
- This changing electrical signal is called Action potential



Before working , Cell is stimulated

Cell sends the signals out

How a cell works

- Action potential will be sent to the connected cells nearby

- After sending the signals, cell will stop working for a while, to wait for its inside ion being stable again, it takes about 1 mSec

In Conclusion

Brain cells have simple working functions, in which the cell gets electrical signals from other cells to stimulate itself to send electrical signals to other cells

Structure of Human Brain and Computer

Brain

-Brain consists of Neuron cells for about 100,000 million cells

-Each Neuron cell does a simple task,
the output signals are waves of electrical signals

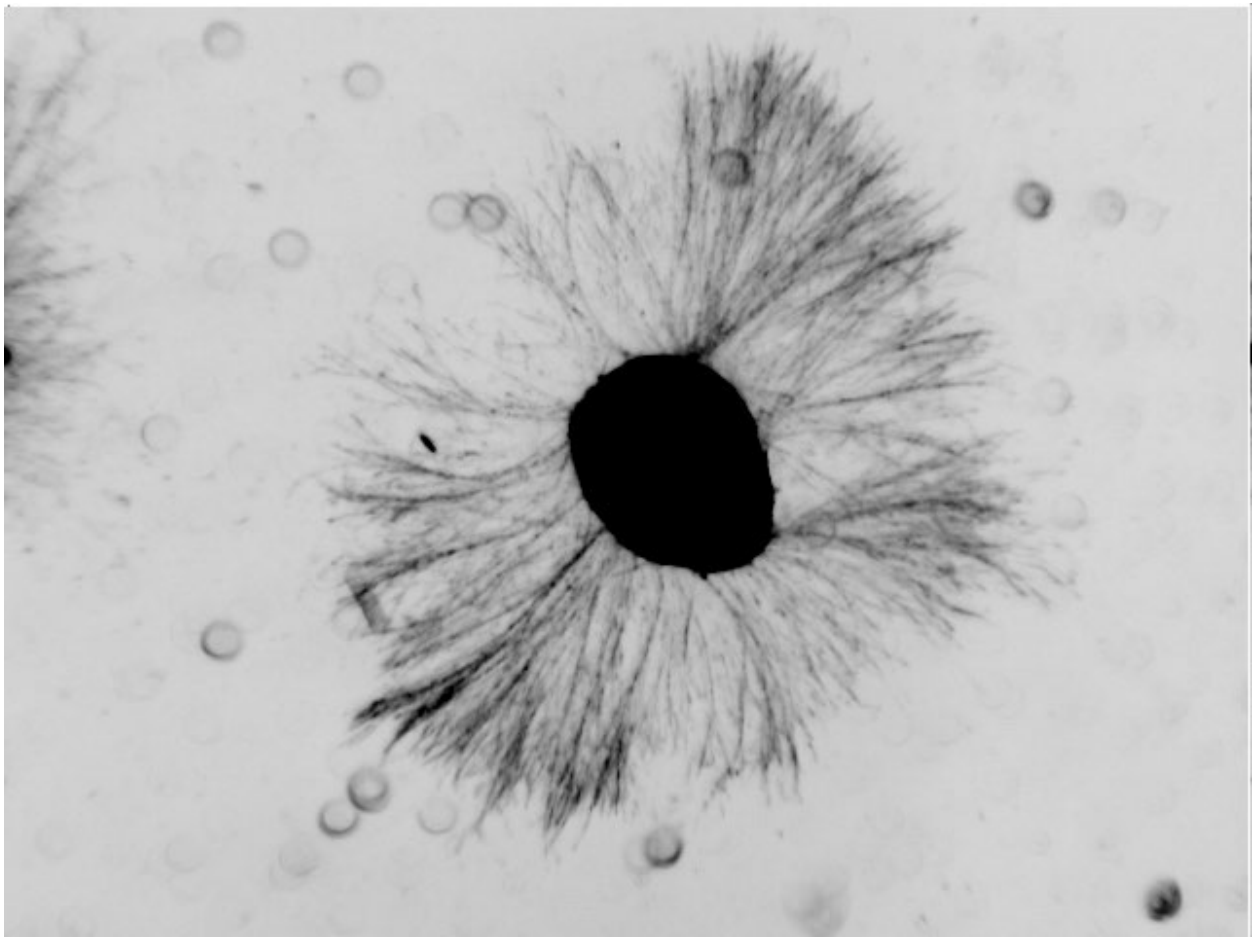
- Normally, each Neuron will connect with other 10,000 Neurons
Some Neural Cells may connect with 200,000 Neurons

- Brain does not have a central memory, but the brain will learn and remember by changing the structure of the brain

-Brain can learn by perform over and over again until those skills can be memorized

Example of neurons

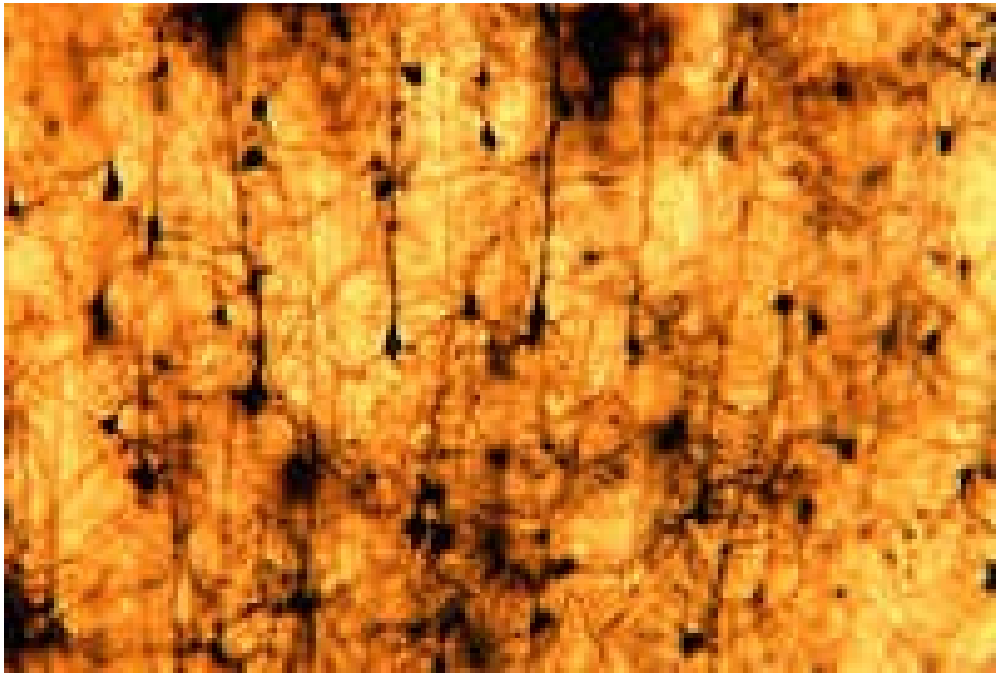
- The cultured retinal explants taken from chick embryos

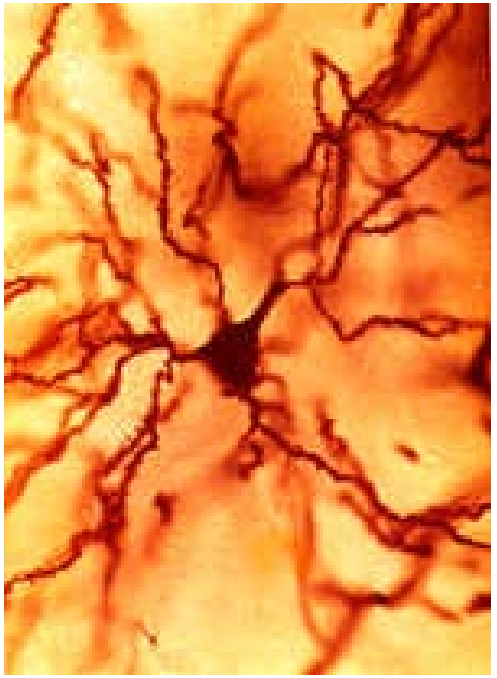


From http://www.ams.sunysb.edu/research/pinezich/neuron_reconstruction/

Example of neurons (continued)

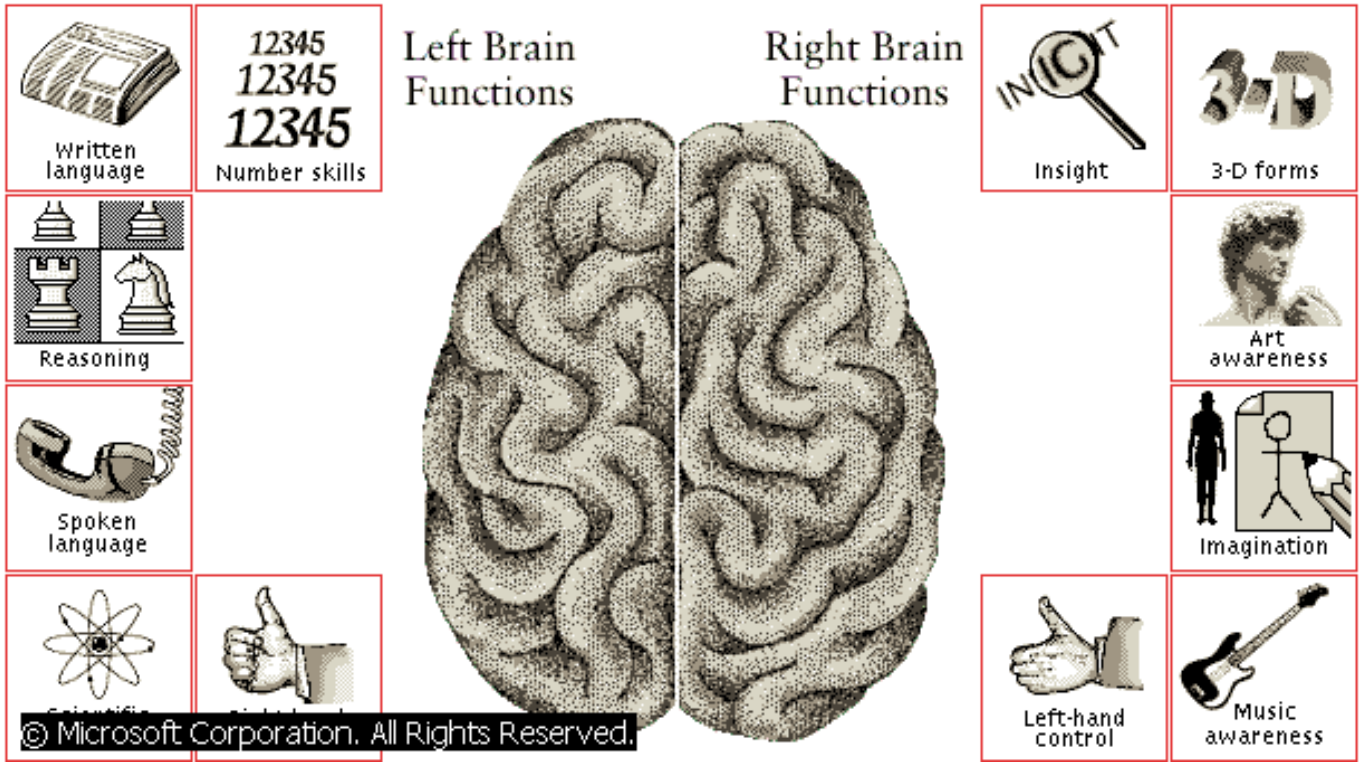
- Neurons located in the cerebral cortex of the hamster.





From http://faculty.washington.edu/chudler/cellpyr.html
(Image courtesy of Dr. James Crandall, Eunice Kennedy Shriver Center)

Brain Works



Scientific skill

Structure of Human Brain and Computer

Computer

-CPU is complex and powerful , each computer has small number of CPUs

Connection between CPUs is simple.

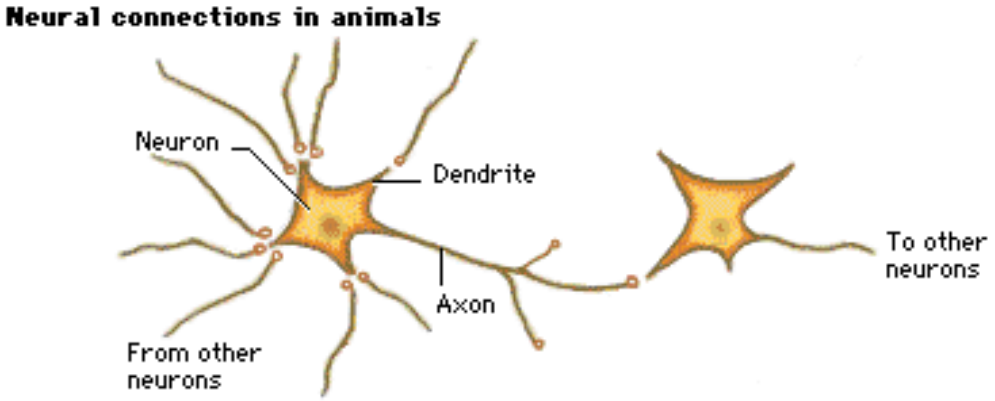
- Central Memory for store programs and data
- Using Programs from instruction set to perform in sequence

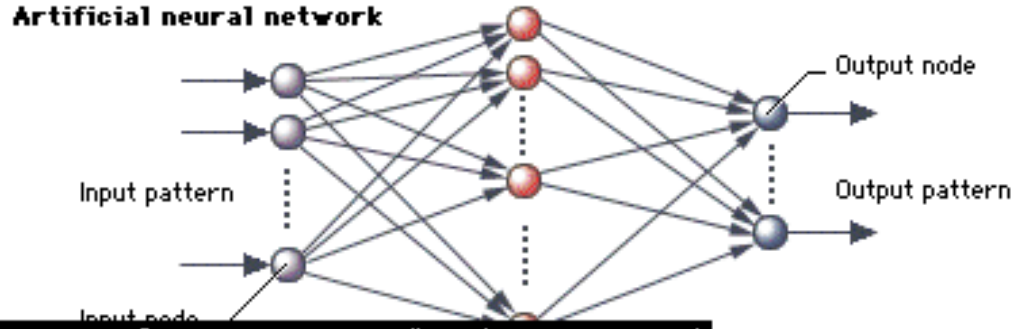
What is Artificial Neural Networks?

A subject about computing by simulating the work of human neural network

Artificial Neural Networks

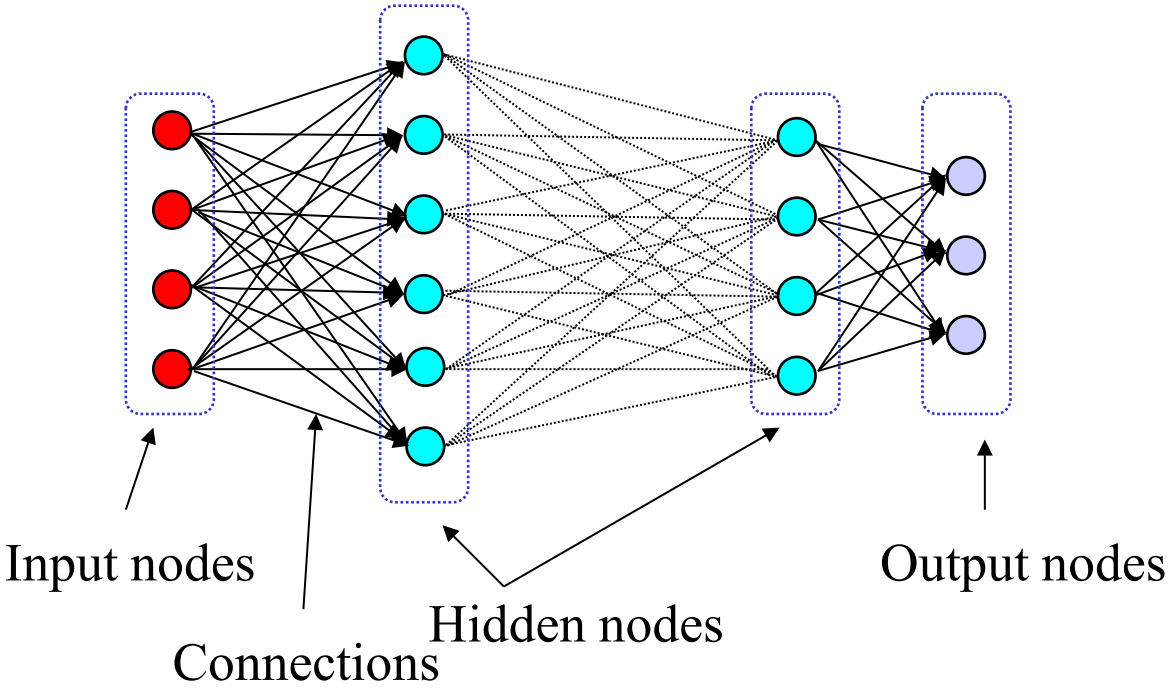
Copy the structure of cells of neural by using huge numbers of simple process connected into a network





© Microsoft Corporation. All Rights Reserved.

An Example



Output for each node

$$y_i = f(w_i^1 x_1 + w_i^2 x_2 + w_i^3 x_3 + \cdots + w_i^m x_m)$$
$$= f(\sum_j w_i^j x_j)$$

X_i = input from other nodes W_i^j = weight for each connection

ANN and Computer

- ANN use concentrating on structure of network of simple process
  Computer uses high performance CPU with small numbers
- Computer executed by programs
  ANN trained by Training set
- Adjusting weight of connections to get minimum training error
- adjusting weight slowly the error will reduced over time
- Normally Simulations are used, the hardware ANN is rarely used

Using ANN

1. Recognition
2. Black Box ---find relationships between inputs and outputs
3. Changeable environments
4. Classifications
5. Predictions

Neural Network Taxonomy

Encoding	Decoding: Feedforward	Decoding: Feedback
Supervised	**Gradient Descent** Least mean square Backpropagation Reinforcement learning	Recurrent Backpropagation
Unsupervised	**Vector Quantization** Self-organizing maps Competitive learning Counter-propagation	**RABAM** Brownian annealing Boltzmann learning ABAM, ART-2, BAM **Adaptive Resonance** ART-1, ART-2

Types of Learning

1. Supervised Learning

Training data will have expected answers to measure the errors for adjust the weights inside the network

2. Unsupervised Learning

No answers to measure the outputs, the network will Rearrange its structure by itself according to received data

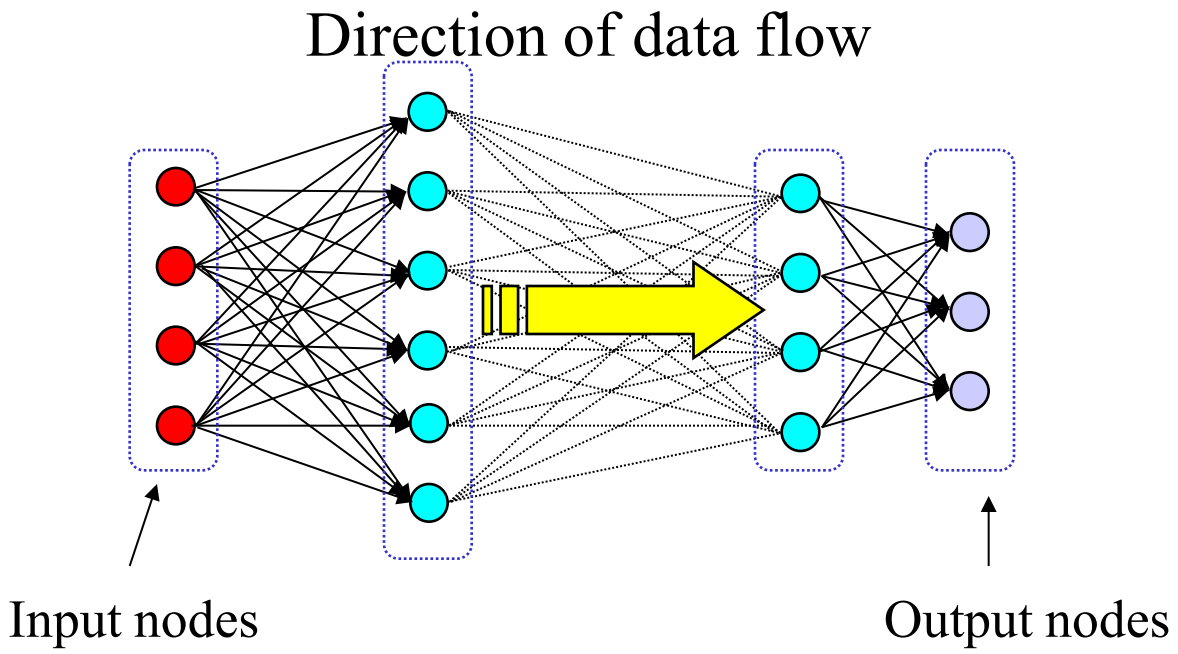
Network Architecture

Feedforward network

Processed data inside the network are sent in the same direction

From input nodes through the inside network until reaching output nodes without flowing data back

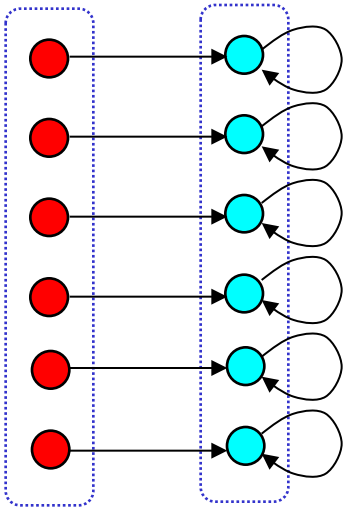
even no connection between nodes inside the same layer



Network Architecture (cont.)

Feedback network

Processed Data inside the network will be feed back to the network many times until getting the solution (sometimes called Recurrent network)



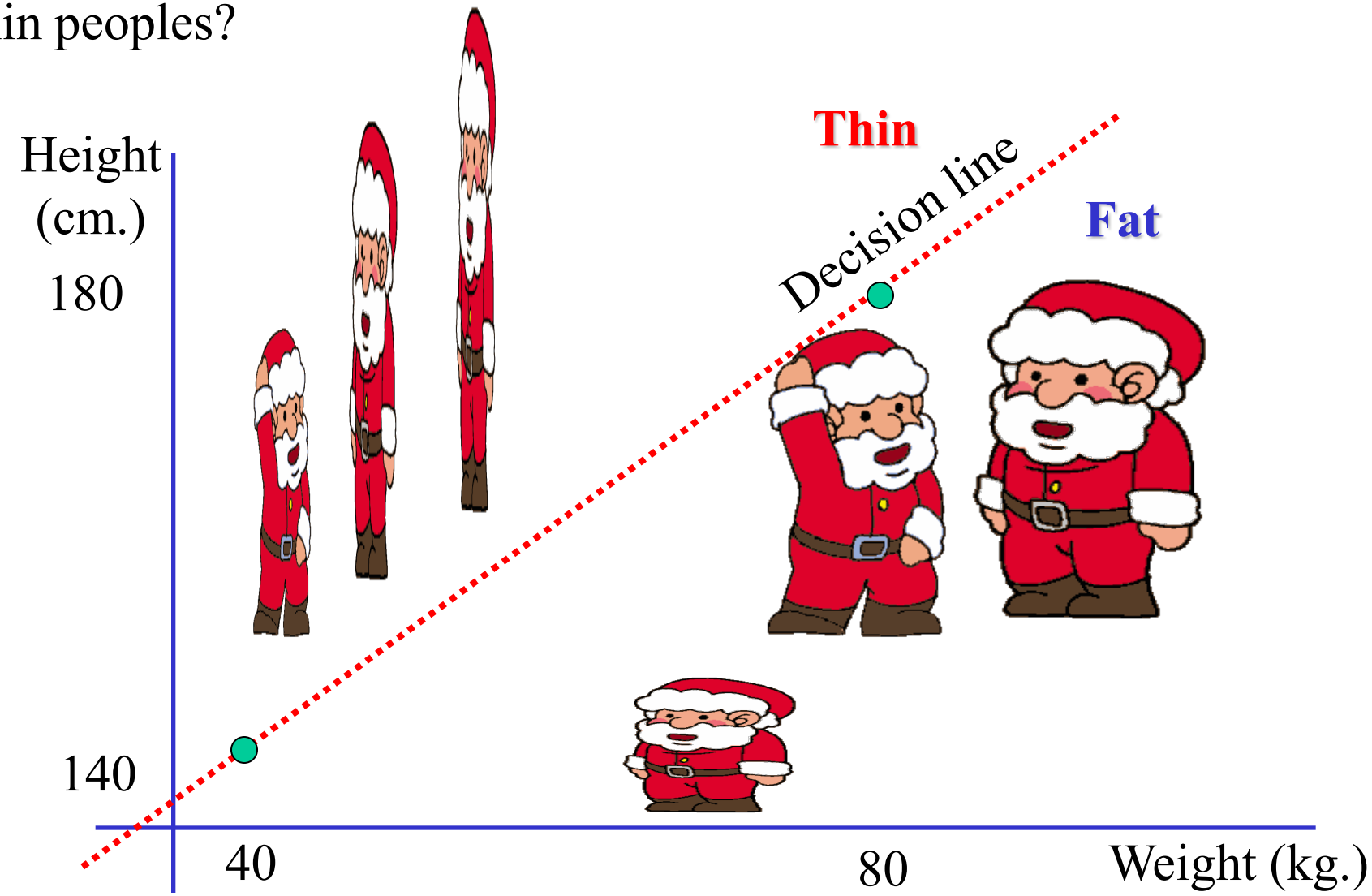
Input nodes Output nodes

Chapter 2

Single Layer Perceptron Networks

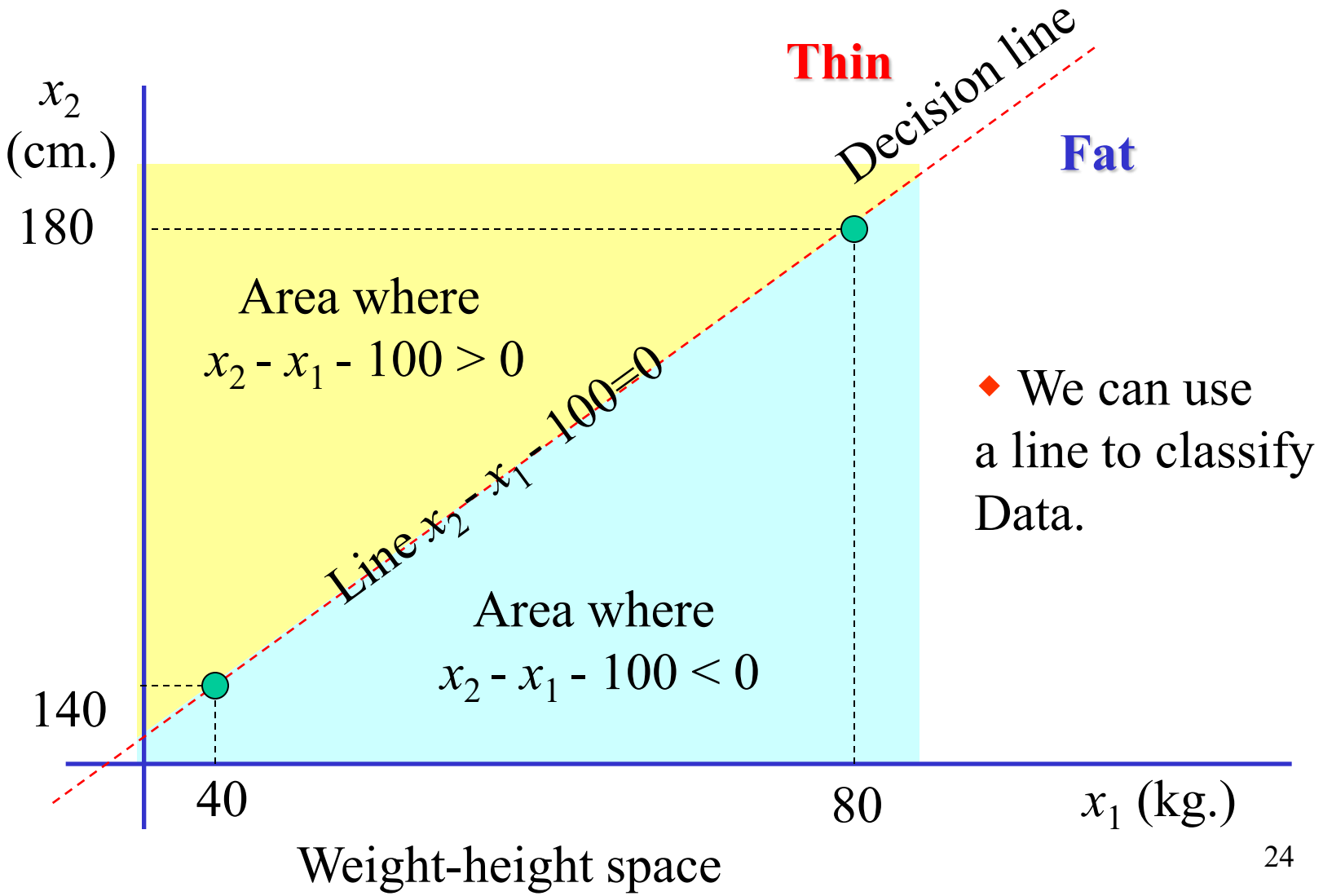
How can we use a mathematical function to classify ?

- Consider a simple problem: How can we classify fat and thin peoples?



How can we use a mathematical function to classify ? (cont.)

- We used 2 inputs, weight (x_1) and height (x_2), to classify "thin" and "fat".



- We can use a line to classify Data.

How can we use a mathematical function to classify ? (cont.)

- We can write the decision function for classifying "thin" and "fat" as follows:

$$y = \begin{cases} 1 \text{ (thin)} & \text{if } x_2 - x_1 - 100 \geq 0 \\ 0 \text{ (fat)} & \text{if } x_2 - x_1 - 100 < 0 \end{cases}$$

or

$$\begin{aligned} y &= g(w_1 x_1 + w_2 x_2 - \mu) \\ &= g(-x_1 + x_2 - 100) \end{aligned}$$

where

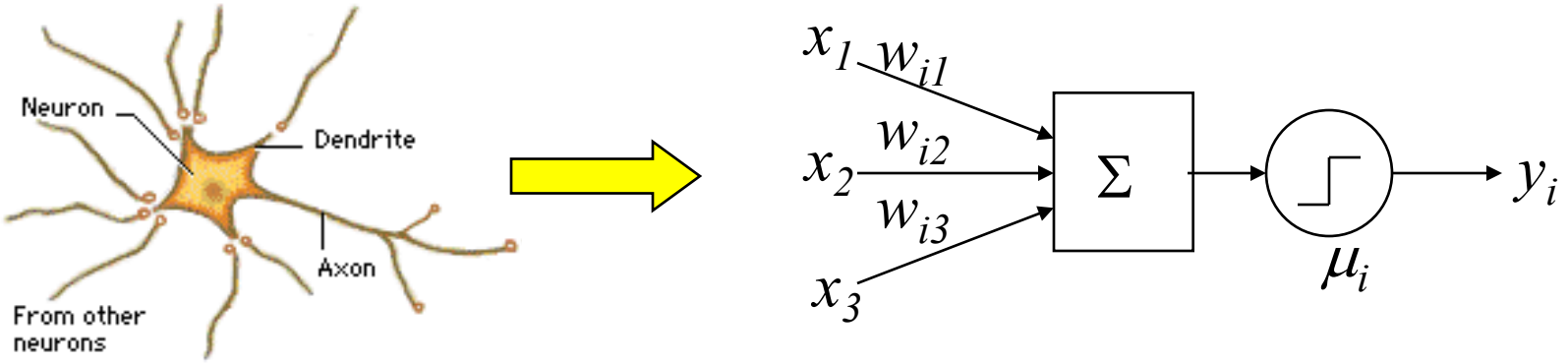
$$g(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise.} \end{cases}$$

Advantage: Universal linear classifier

Problem: For a particular problem, **how can we choose suitable weights *w* and μ of the function ?**

Supervised Learning: Perceptron Networks

Rosenblatt and his team invent the idea of Perceptron
In 1962 inspired by the model of neural cells of McCulloch-Pitts



McCulloch-Pitts model

$$y_i = \theta(w_{i1}x_1 + w_{i2}x_2 + w_{i3}x_3 - \mu_i)$$
$$= \theta(\sum_j w_{ij}x_j - \mu_i)$$

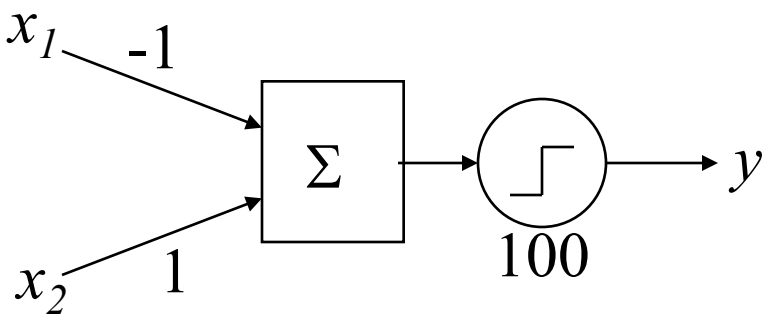
Supervised Learning: Perceptron Networks (cont.)

- The decision function to classify "thin" and "fat" using the McCulloch-Pitts model:

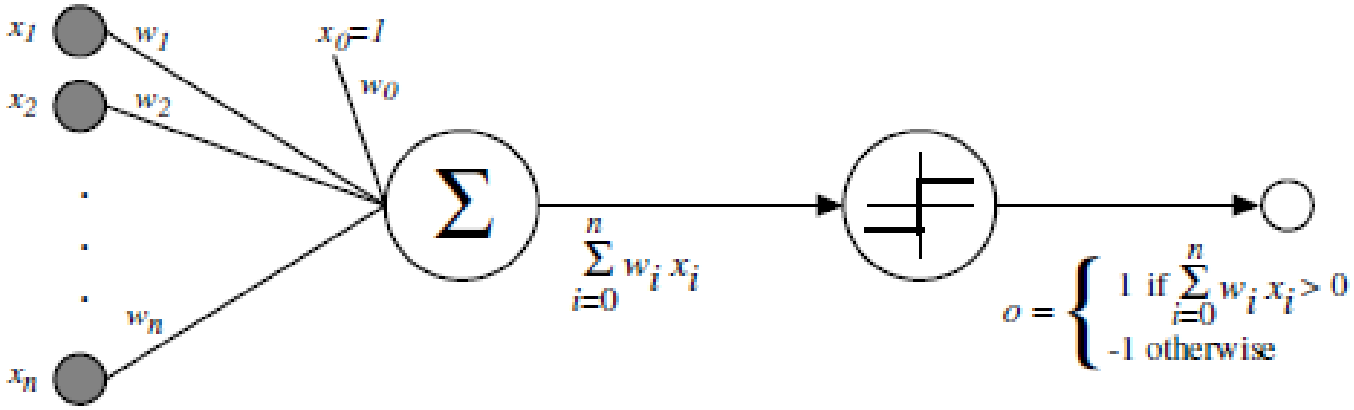
$$y = g(w_1 x_1 + w_2 x_2 - \mu)$$
$$= g(-x_1 + x_2 - 100)$$

where

$$g(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise.} \end{cases}$$



Perceptron

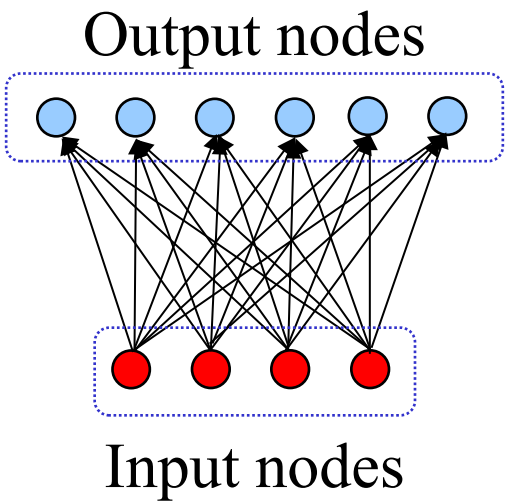


$$o(x_1, \ldots, x_n) = \begin{cases} 1 & \text{if } w_0 + w_1 x_1 + \cdots + w_n x_n > 0 \\ -1 & \text{otherwise.} \end{cases}$$

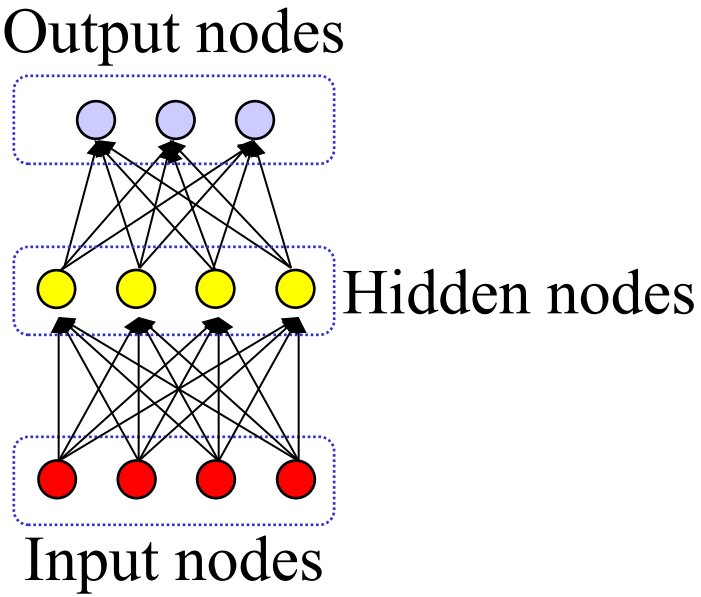
Sometimes we'll use simpler vector notation:

$$o(\vec{x}) = \begin{cases} 1 & \text{if } \vec{w} \cdot \vec{x} > 0 \\ -1 & \text{otherwise.} \end{cases}$$

Perceptron Networks

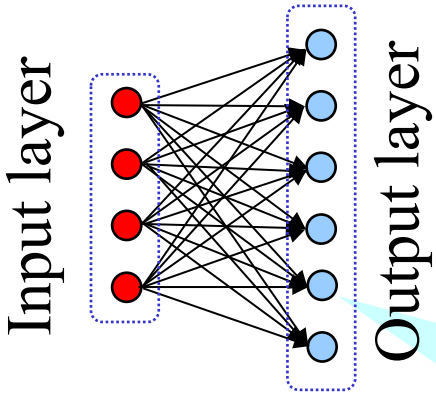


Single layer percentron network



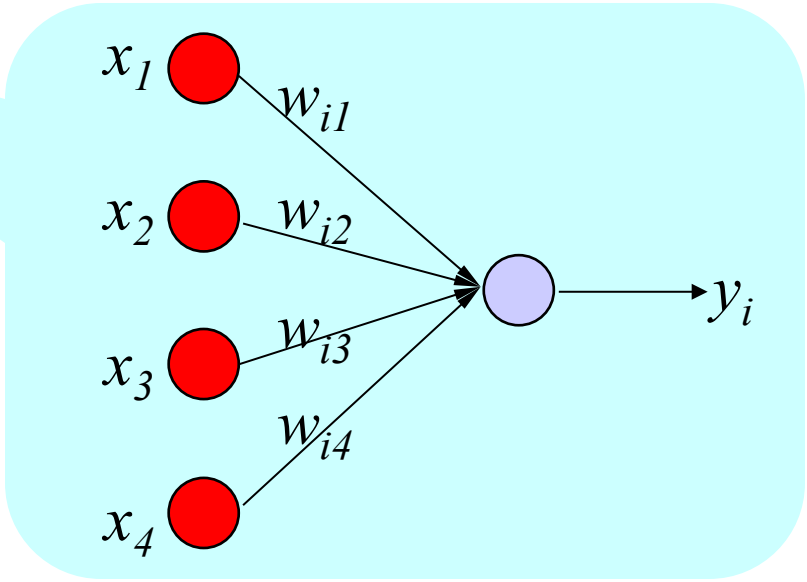
Multilayer percentron network
(this case: 2 layers)

A Single Layer Perceptron Network



For each node, the output is given by

$$y_i = g\left(\sum_{j=1}^{N} w_{ij} x_j - \mu_i\right)$$



w_{ij} = Connection weight of branch *(i,j)*

x_j = Input data from node *j* in the input layer

μ_i = Threshold value of node *i* in the output layer

g = ***Activation function***

Single Layer Perceptron Networks (cont.)

1. The number of input nodes

   depend on the number of components of input data

2. Activation function can be defined from the characteristic of Output

if outputs are either "yes" or "no" we use *Threshold function*

$$f(x) = \begin{cases} 1 & \text{if } x \geq T \\ 0 & \text{if } x < T \end{cases}$$

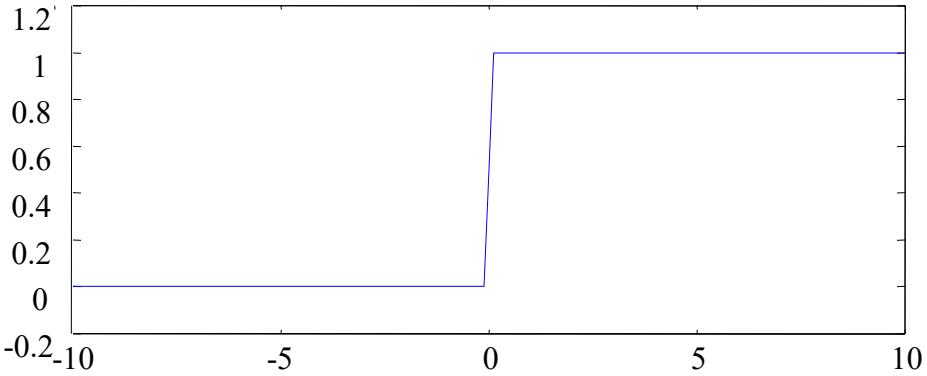
T=Threshold level

if outputs are continuous numbers, we need continuous function as *Sigmoid function*

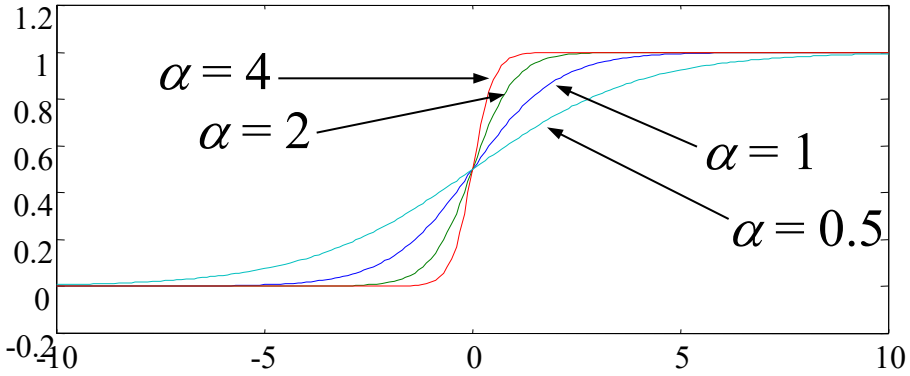
$$f(x) = \frac{1}{1 + e^{-\alpha x}}$$

Activation function

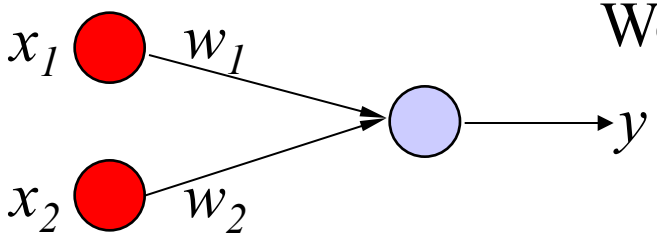
Threshold function (T=0)



Sigmoid function

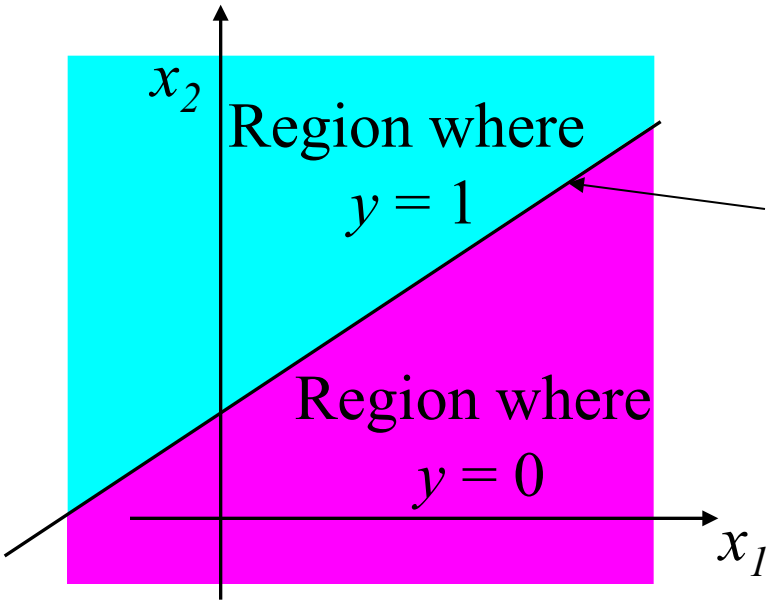


How a single layer perceptron works



Assume that we have perceptron of 2 input node and activation function as threshold function
We will have Binary output

$$y = \begin{cases} 1 & \text{if} \quad w_1x_1 + w_2x_2 - \theta \geq 0 \\ 0 & \text{if} \quad w_1x_1 + w_2x_2 - \theta < 0 \end{cases}$$



line $L =$ $w_1x_1 + w_2x_2 - \theta = 0$

- if (x_1,x_2) is over the line L, we get $y = 1$
- if (x_1,x_2) is under the line L, we get $y = 0$

So we call this line L as
Decision function or
Boundary Function

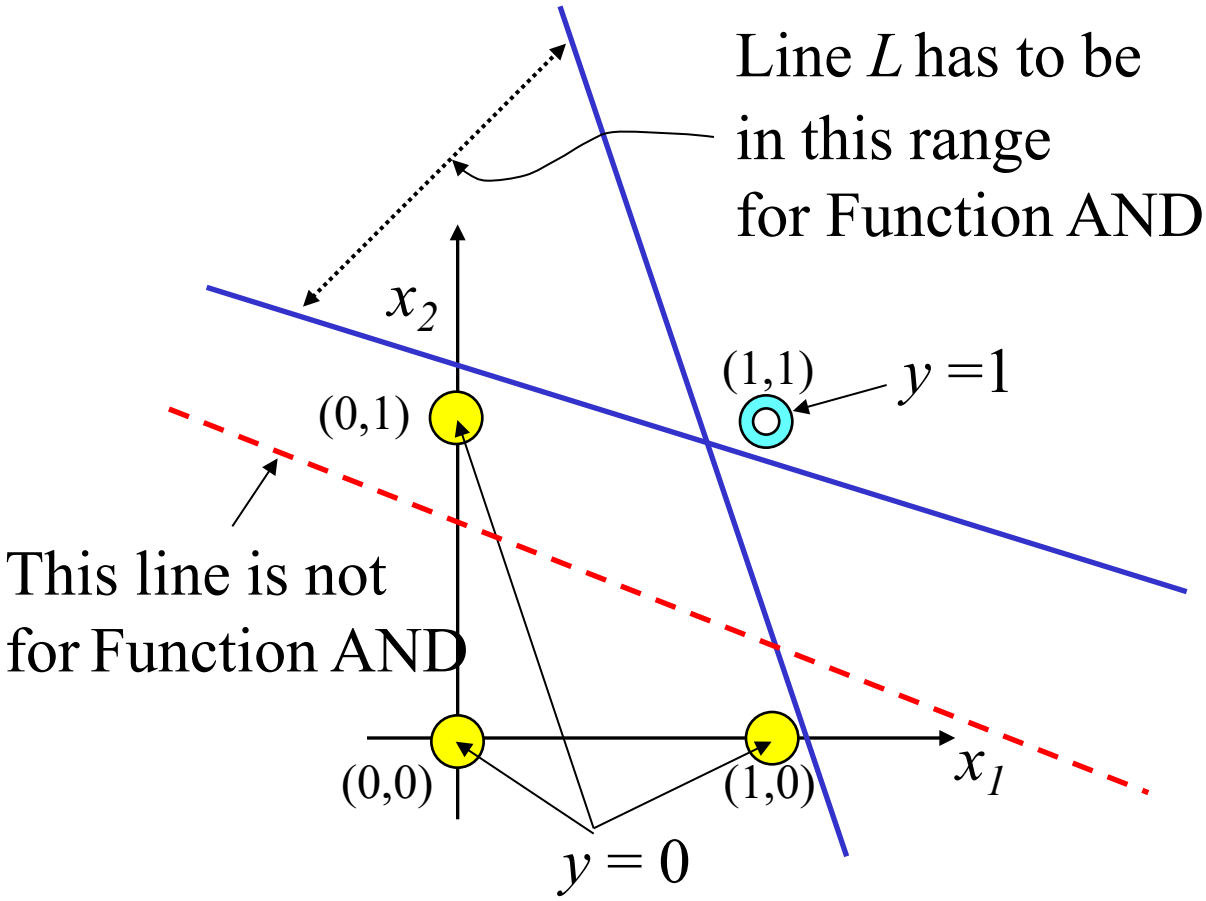
How a single layer perceptron works (cont.)

Slope and Position of line $L : w_1x_1+w_2x_2 - q = 0$ depend on parameters w_1, w_2, and q , *we have to adjust parameters for a right line*

Example

Function AND

x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1



Learning Algorithm: Training Perceptron Networks

Adjusting The Networks: adjust parameters in the way to Reduce the error

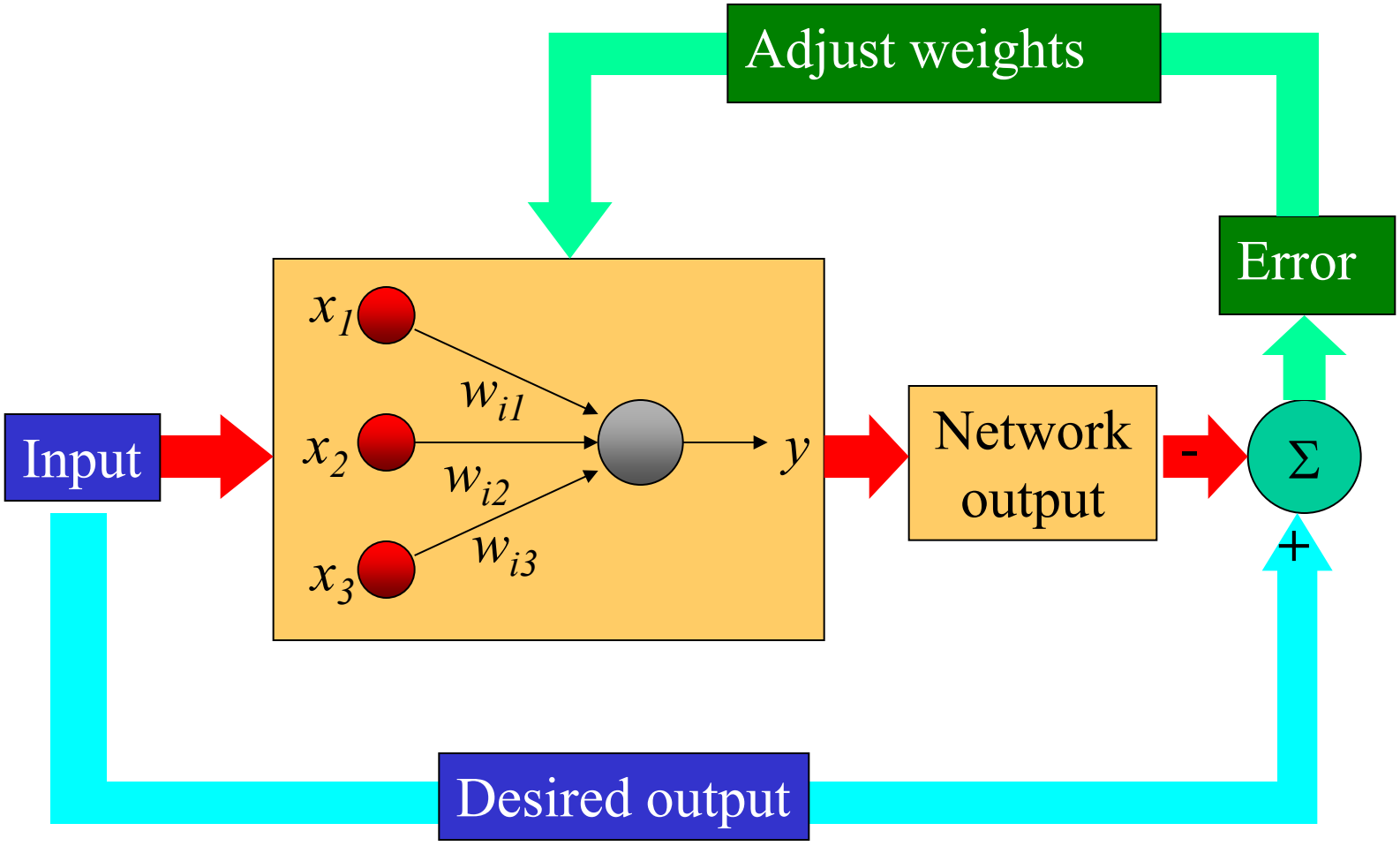
$$\varepsilon = y - \hat{y}$$

ε = Error, y = Desired output, $\hat{y}$ = Network output

Step of Training the network

1. feed Input to Network	Input (x_1, x_2) Desired output y
2. computeNetwork Output	$\hat{y} = f(w_1 x_1 + w_2 x_2 - \theta)$
3. compute Error	$\varepsilon = y - \hat{y}$
4. adjust all weights	$w^{new} = w^{old} + \Delta w, \quad \theta^{new} = \theta^{old} + \Delta\theta$
5. do step 1 until Error is less than a threshold	$\lvert\varepsilon\rvert < t$

Learning Algorithm: Training Perceptron Networks (cont.)



Learning Algorithm: Training Perceptron Networks (cont.)

Training Perceptron Networks by adjusting weights !

Formula for adjusting weight

$$\Delta w_i = \alpha(y - \hat{y})x_i$$

$$\Delta\theta = -\alpha(y - \hat{y})$$

α = Learning rate

feeding input-output pairs for training have to be complete

1. Feed all pairs accordingly in the table or
2. Feed randomly

Perceptron training rule

$$w_i \leftarrow w_i + \Delta w_i$$

where

$$\Delta w_i = \eta(t - o)x_i$$

Where:

- $t = c(\vec{x})$ is target value
- o is perceptron output
- η is small constant (e.g., .1) called *learning rate*

Learning Algorithm: Training Perceptron Networks (cont.)

Example

Function AND

x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

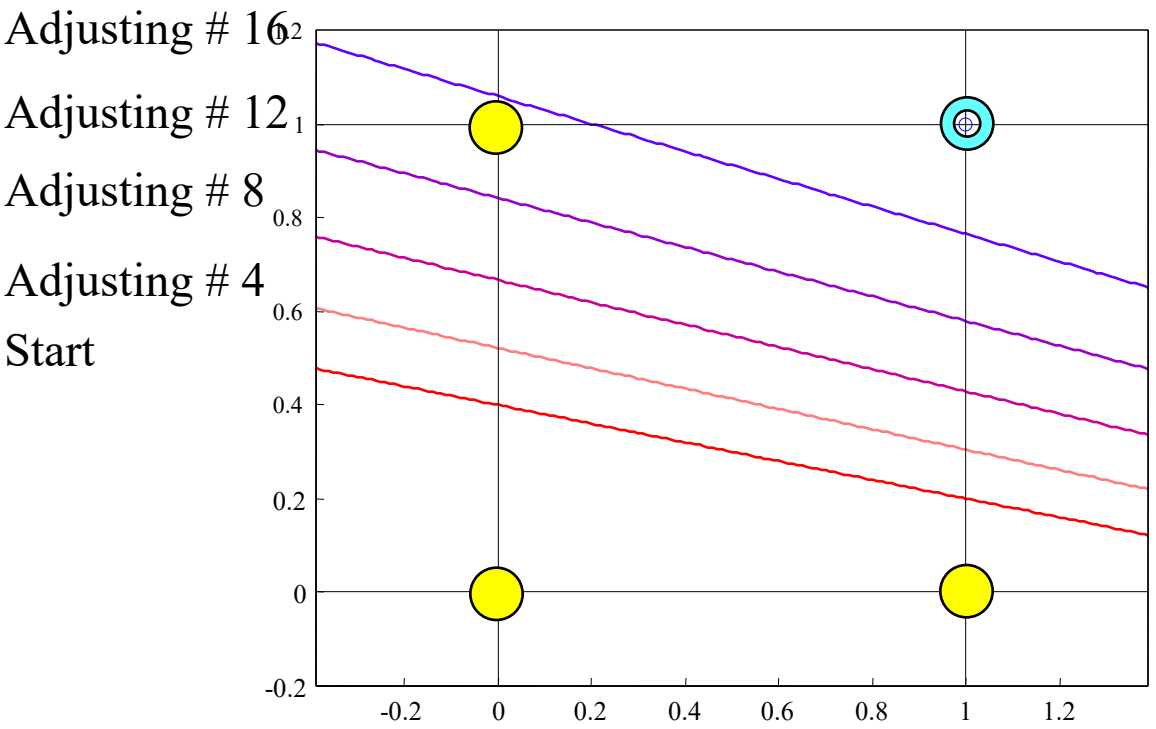
Initial Values

$w_1 = 0.5$

$w_2 = 2.5$

$\theta = 1.0$

$\alpha = 0.2$



Learning Algorithm: Training Perceptron Networks (cont.)

n	x_1	x_2	y	y^	Err	W_1 New	Δw_1	W_2 New	Δw_2	Θ New
0						0.5		2.5		1
1	0	0	0	0	0	0.5	0	2.5	0	1
2	0	1	0	1	-1	0.5	0	2.3	-0.2	1.2
3	1	0	0	0	0	0.5	0	2.3	0	1.2
4	1	1	1	1	0	0.5	0	2.3	0	1.2
5	0	0	0	0	0	0.5	0	2.3	0	1.2
6	0	1	0	1	-1	0.5	0	2.1	-0.2	1.4
7	1	0	0	0	0	0.5	0	2.1	0	1.4
8	1	1	1	1	0	0.5	0	2.1	0	1.4
9	0	0	0	0	0	0.5	0	2.1	0	1.4
10	0	1	0	1	-1	0.5	0	1.9	-0.2	1.6
…	…	…	…	…	…	…	…	…	…	…

$$\hat{y} = f(w_1 x_1 + w_2 x_2 - \theta)$$

$$\varepsilon = y - \hat{y}$$

$$\Delta w_i = \alpha(y - \hat{y}) x_i$$

$$\Delta\theta = -\alpha(y - \hat{y})$$

$$w^{new} = w^{old} + \Delta w$$

$$\theta^{new} = \theta^{old} + \Delta\theta$$

Classification Data

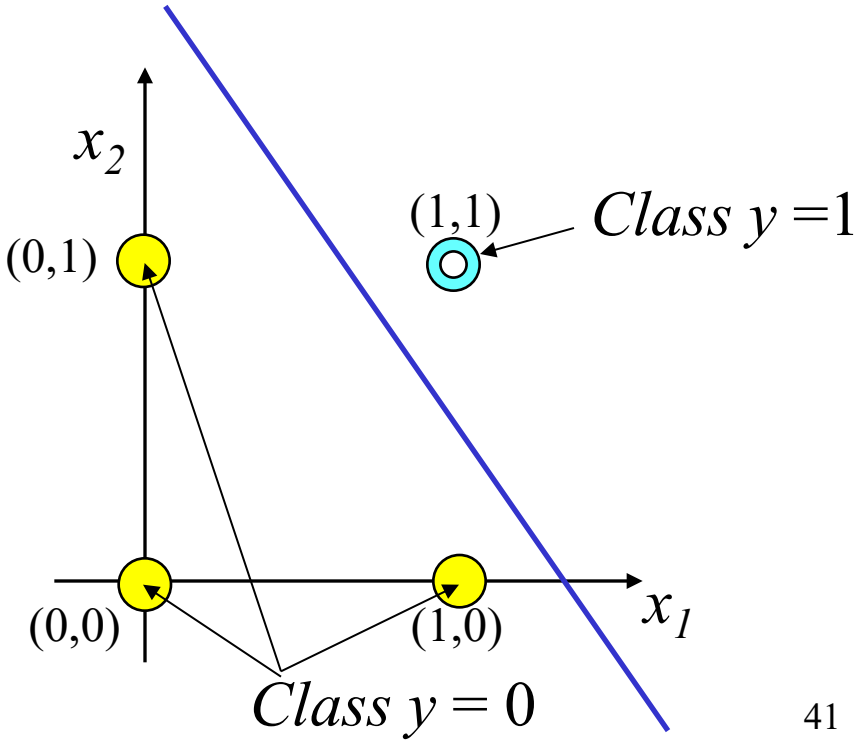
Using a single layer perceptron network for classifying data

- We call input having many dimensions as **input pattern** or input vector
- Output should have many **classes**

Features of data that can be used with a single layer perceptron Network to classify

-feature space (input domain)

2 dimensions, each class has to be able to separate other classes by using a line.



Classification Data (cont.)

Function XOR

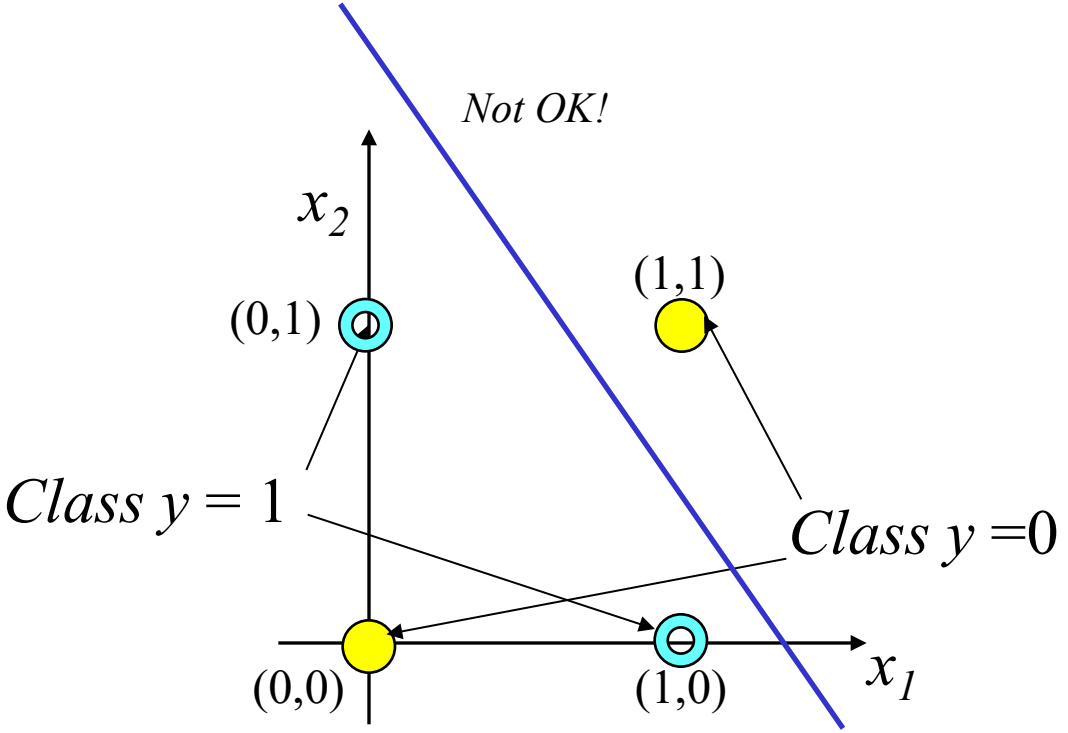
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

An example of a single layer perceptron

That can not be used

Function XOR we can not use a line

To separate Class y=0 and Class y=1



Higher Dimension Feature Space

In the case that input pattern has component more than 1 component, output of perceptron will be…

$$y = \begin{cases} 1 & \text{if} \quad w_1 x_1 + w_2 x_2 + \cdots + w_N x_N - \theta \geq 0 \\ 0 & \text{if} \quad w_1 x_1 + w_2 x_2 + \cdots + w_N x_N - \theta < 0 \end{cases}$$

We will have decision function as

$$w_1 x_1 + w_2 x_2 + \cdots + w_N x_N - \theta = 0$$

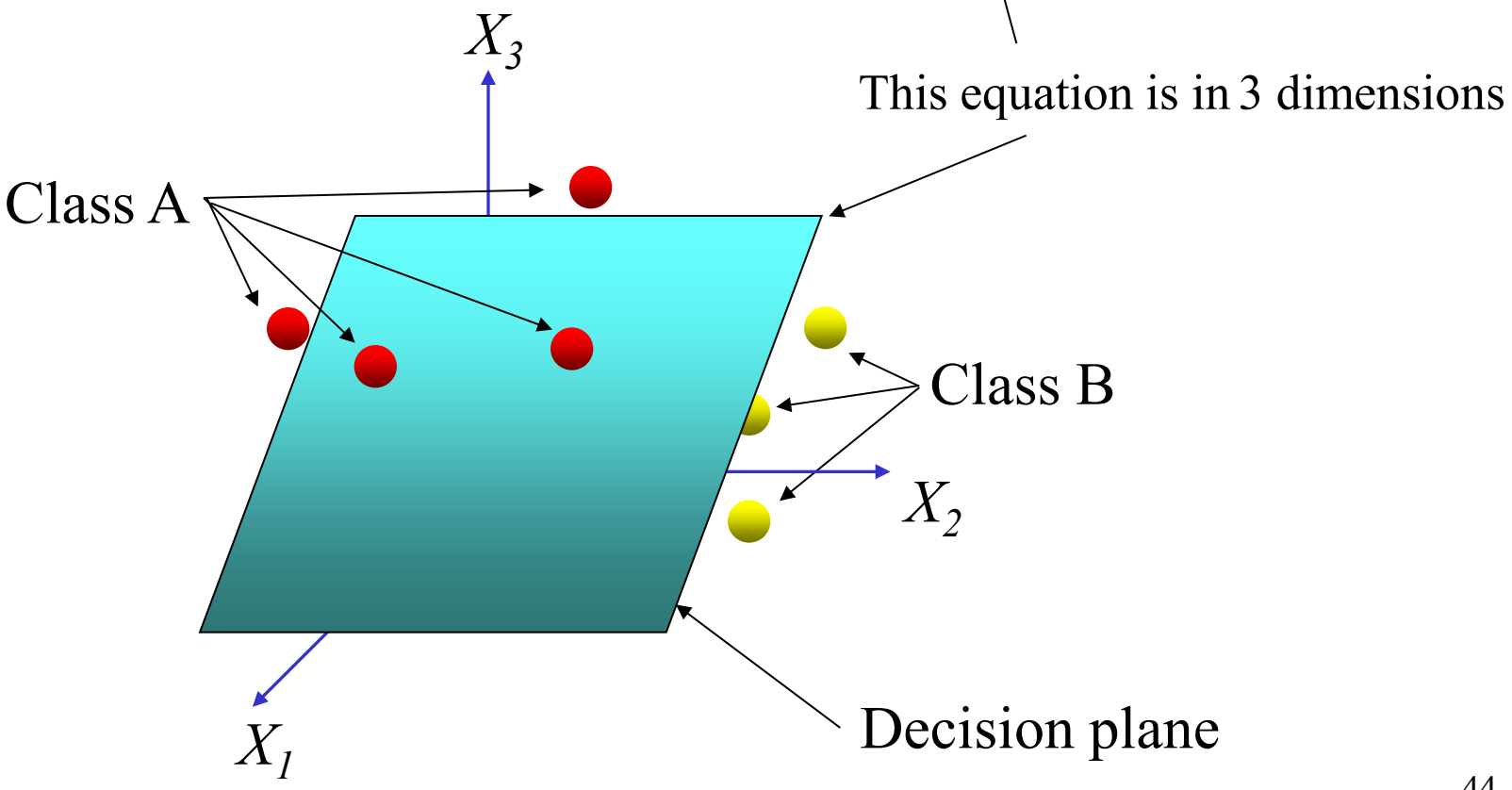
this equation is a multi dimension (hyperplane)

In the case of input 2 dimensions, decision function will be a line

Higher Dimension Feature Space (cont.)

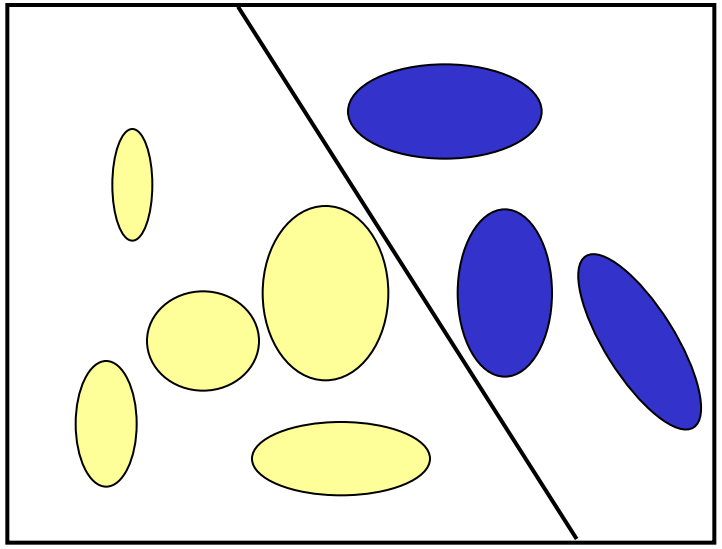
In the case of input 3 dimensions, we will have a decision function

$$w_1x_1 + w_2x_2 + w_3x_3 - \theta = 0$$

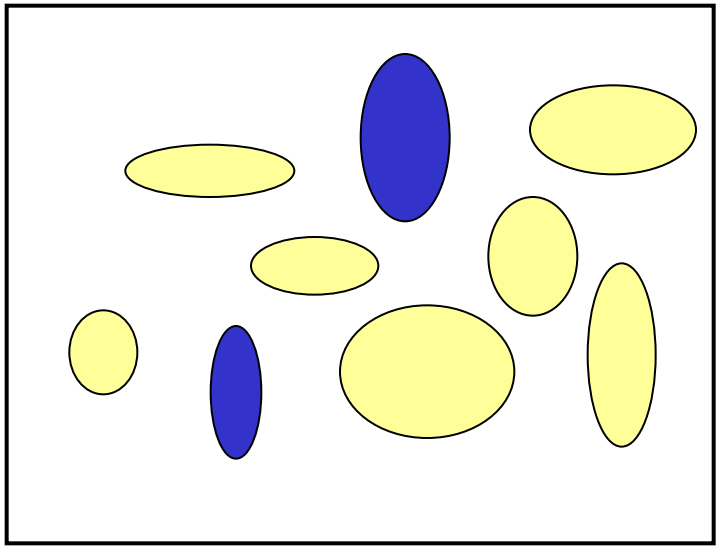


Linear Separability

- In the case of a single layer perceptron we have hyperplane as a decision function
- if input pattern for each class can be separated other classes by using hyperplane we call this data having a linearly separable



Linearly separable



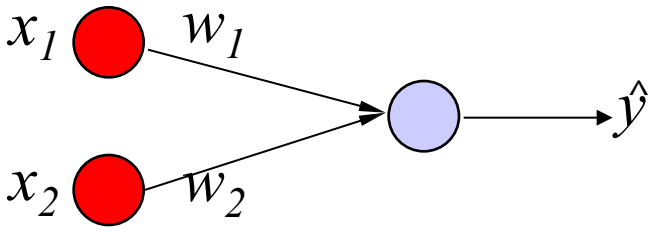
Not linearly separable

(limitation of a single layer perceptron)

How can we adjust weights?

considering we have a function $y = x_1 + 2x_2$

And would like to use a single layer perceptron to estimate this function



with output in the form $\hat{y} = w_1 x_1 + w_2 x_2$

(in this case, activation function is an identity function $f(x) = x$)

-We have to adjust w_1 and w_2 that makes y close^
to y as close as it can.

Gradient Descent

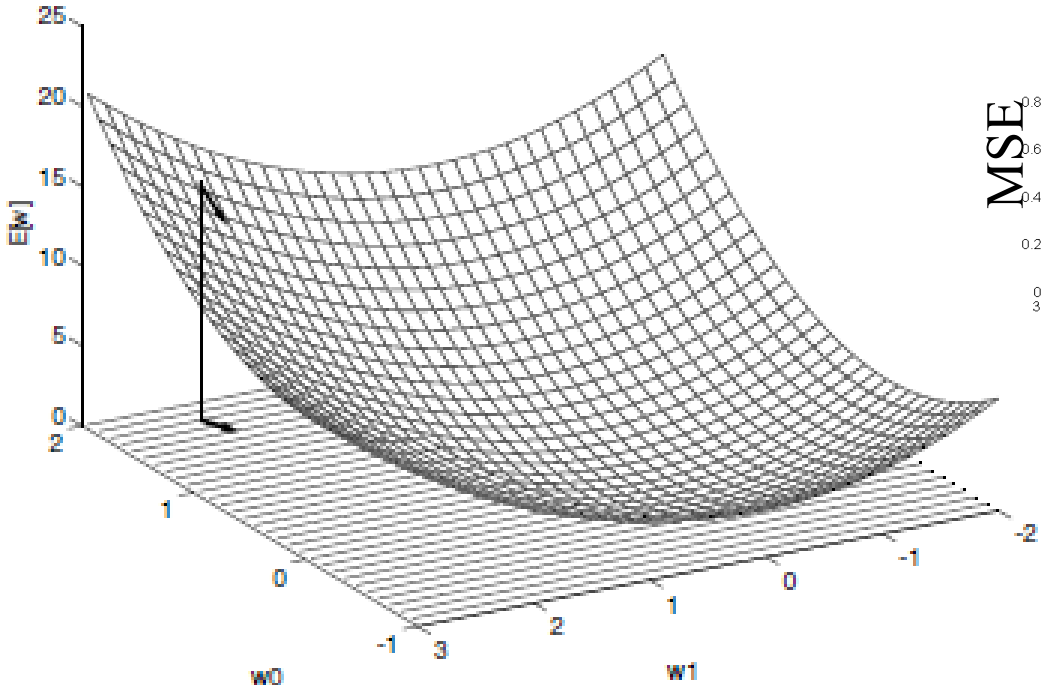
To understand, consider simpler *linear unit*, where

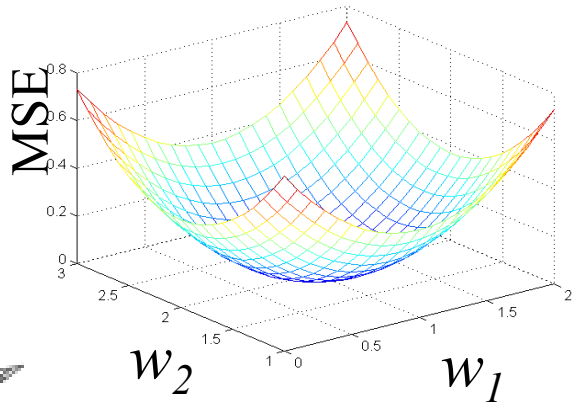
$$o = w_0 + w_1 x_1 + \cdots + w_n x_n$$

Let's learn w_i's that minimize the squared error

$$E[\vec{w}] \equiv \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2$$

Where D is set of training examples





Gradient

$$\nabla E[\vec{w}] \equiv \left[\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \cdots \frac{\partial E}{\partial w_n}\right]$$

Training rule:

$$\Delta \vec{w} = -\eta \nabla E[\vec{w}]$$

i.e.,

$$\Delta w_i = -\eta \frac{\partial E}{\partial w_i}$$

$$
\begin{aligned}
\frac{\partial E}{\partial w_i} &= \frac{\partial}{\partial w_i}\frac{1}{2}\sum_d (t_d - o_d)^2 \\
&= \frac{1}{2}\sum_d \frac{\partial}{\partial w_i}(t_d - o_d)^2 \\
&= \frac{1}{2}\sum_d 2(t_d - o_d)\frac{\partial}{\partial w_i}(t_d - o_d) \\
&= \sum_d (t_d - o_d)\frac{\partial}{\partial w_i}(t_d - \vec{w}\cdot\vec{x_d}) \\
\frac{\partial E}{\partial w_i} &= \sum_d (t_d - o_d)(-x_{i,d})
\end{aligned}
$$

GRADIENT-DESCENT($training_examples, \eta$)

Each training example is a pair of the form $\langle \vec{x}, t \rangle$, *where* $\vec{x}$ *is the vector of input values, and* t *is the target output value.* η *is the learning rate (e.g., .05).*

- Initialize each w_i to some small random value
- Until the termination condition is met, Do
  - Initialize each Δw_i to zero.
  - For each $\langle \vec{x}, t \rangle$ in *training_examples*, Do
    - Input the instance $\vec{x}$ to the unit and compute the output o
    - For each linear unit weight w_i, Do

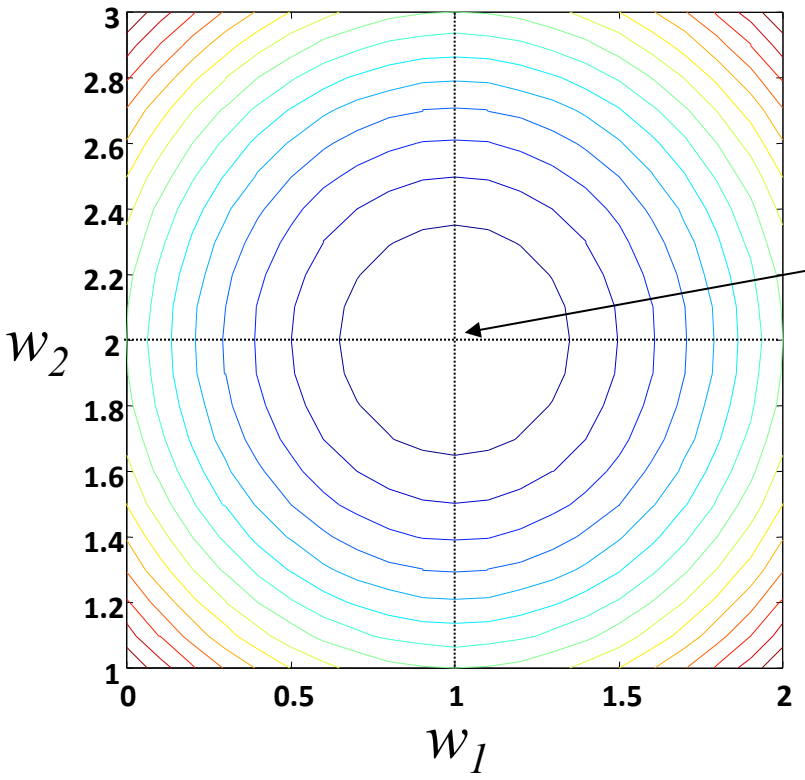
$$\Delta w_i \leftarrow \Delta w_i + \eta(t - o)x_i$$

  - For each linear unit weight w_i, Do

$$w_i \leftarrow w_i + \Delta w_i$$

How can we adjust weights? (cont.)

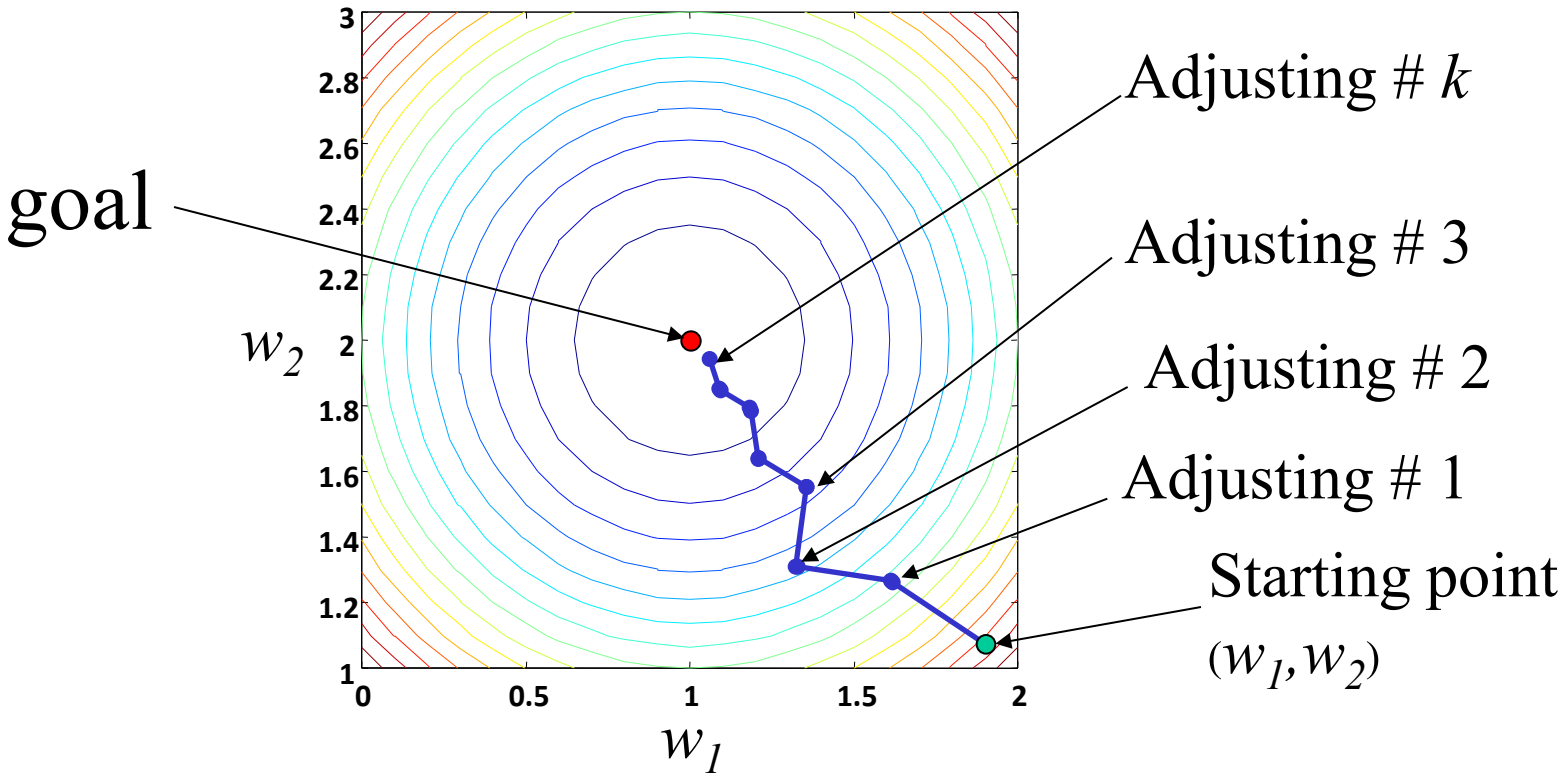
Mean square error ε^2 as a function of w_1 and w_2



We have to adjust w_1 and w_2 to converse to the minimum point on error surface.

How can we adjust weights? (cont.)

This is how adjusting w_1 and w_2 to the minimum point on error surface.



Gradient Descent Method

We want to adjust w_1 and w_2 to converse to the minimum point of error surface.

In this situation, we can set the problem as follow

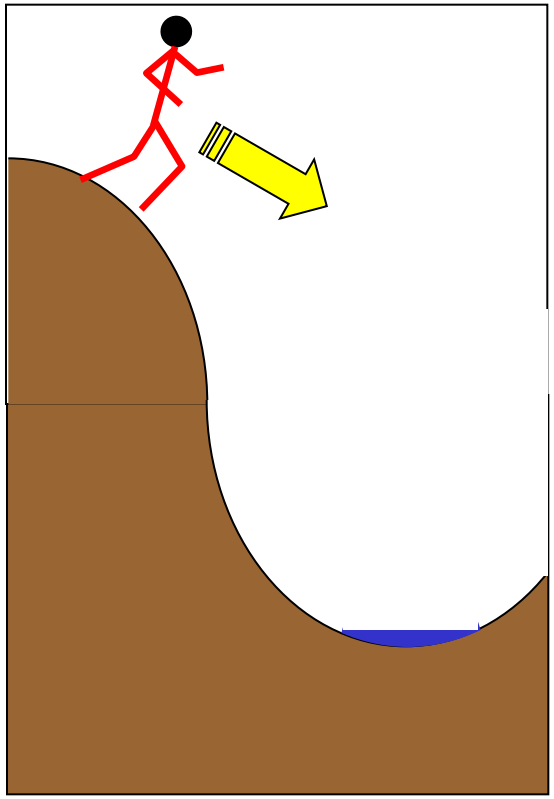
Questions

1. Error surface is a valley which has a blinded man stand on the top
2. Having a pool on the bottom
3. This man want to walk to the pool
4. He know only the slope of the floor he stand
5. The question is how this man walk to the pool

Answer

Walk to direction that the steepest slope down to the pool

Gradient Descent Method (cont.)



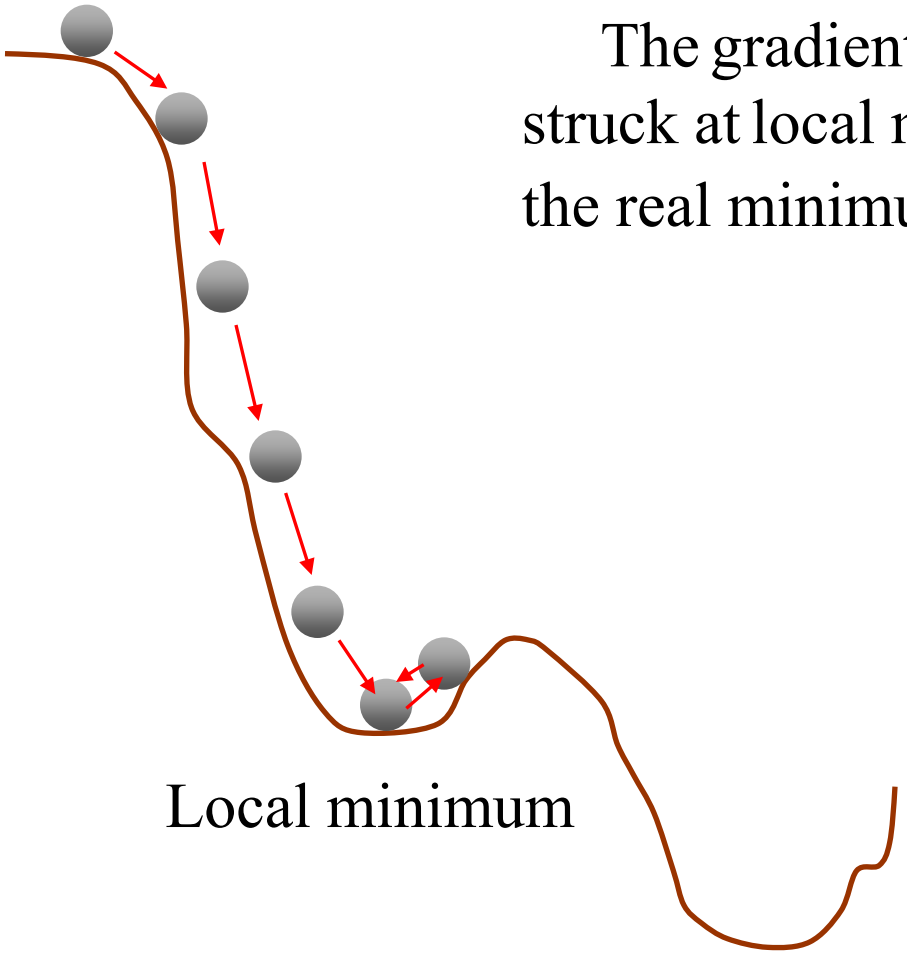
Moving down the hill in the direction that has the highest slope is called Gradient Descent Method

1. compute gradient of the error surface on the current position (w_1,w_2)
   gradient will point the highest slope to the top of the hill

2. Move to the opposite direction of gradient for a short distance (by adjusting w_1,w_2)

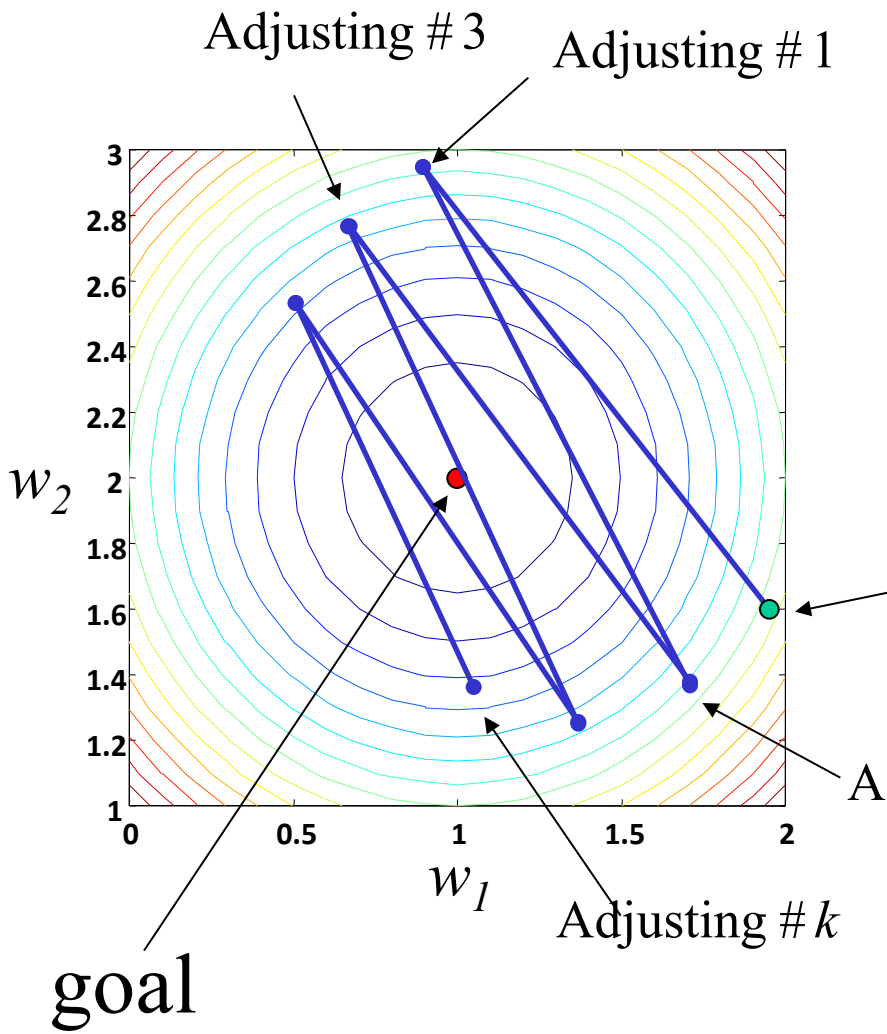
3. Go to step 1 until reach the minimum point

Weak point of Gradient Descent Method



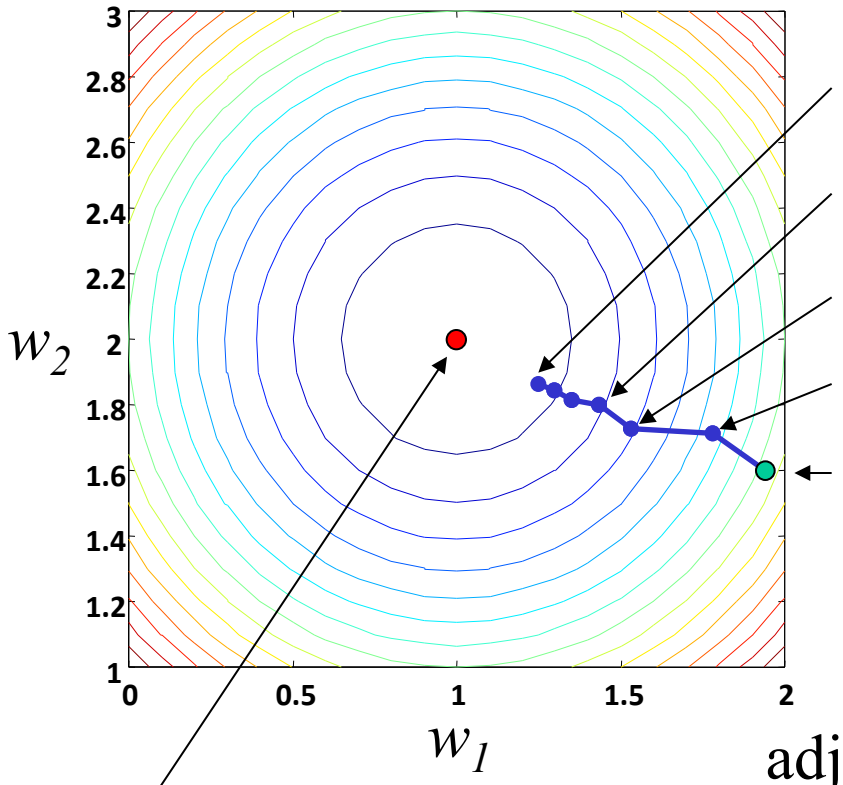
The gradient descent method may get struck at local minima which is not the real minimum point

Why we need to adjust weight using a small step



adjusting weight by using high learning rate, it will make network slowly converse to a minimum point and unstable

Why we need to adjust weight using a small step (cont.)



Adjusting # k

Adjusting # 3

Adjusting # 2

Adjusting # 1

← Starting for (w_1, w_2)

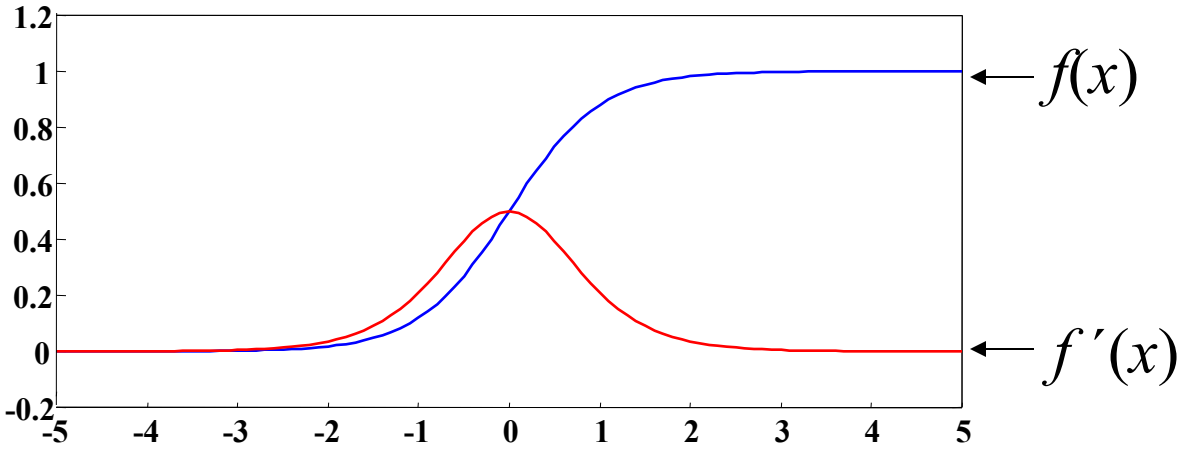
goal

adjusting weight a little by little

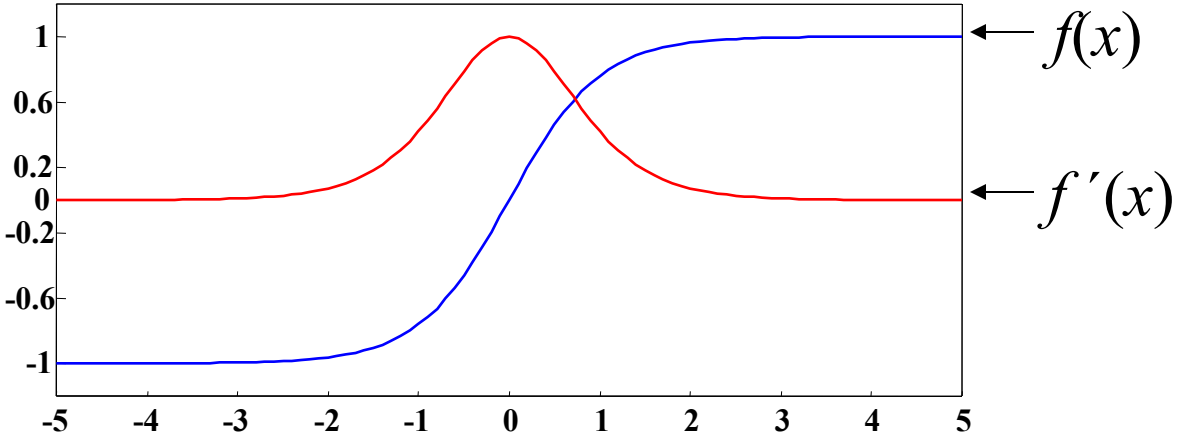
It makes network adjusting to the minimum point better and reducing unstable

Adjusting Weights for a Nonlinear Unit (cont.)

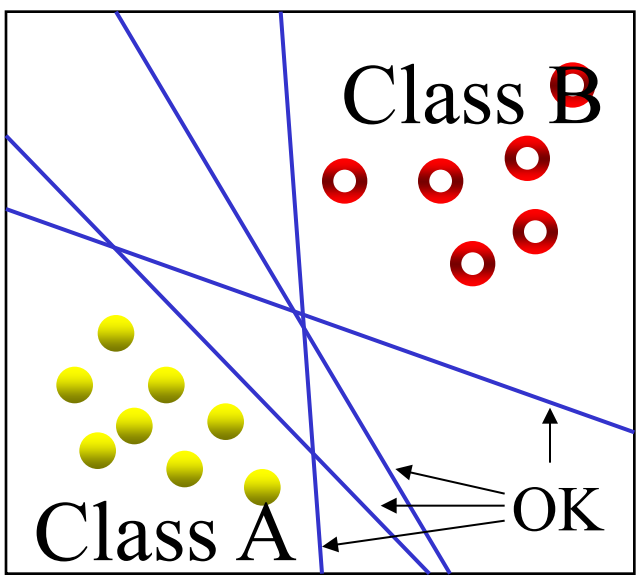
Sigmoid function ($\beta = 1$)



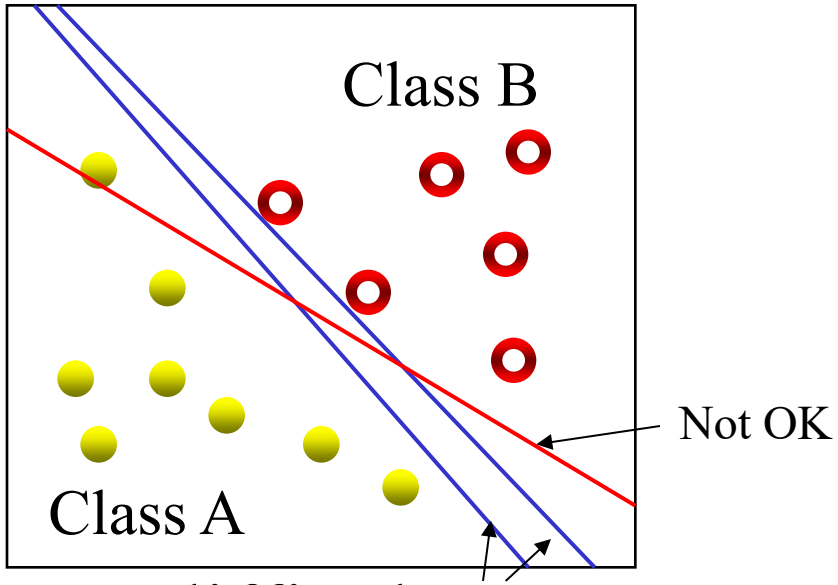
tanh function ($\beta = 1$)



How difficult the data for the network to learn



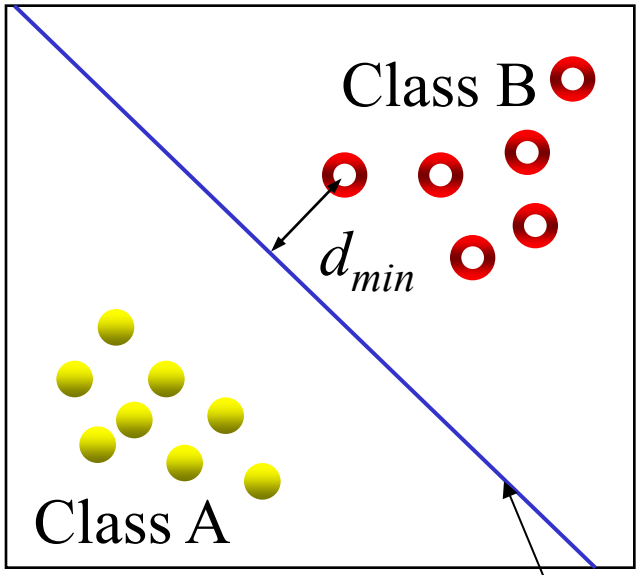
easy



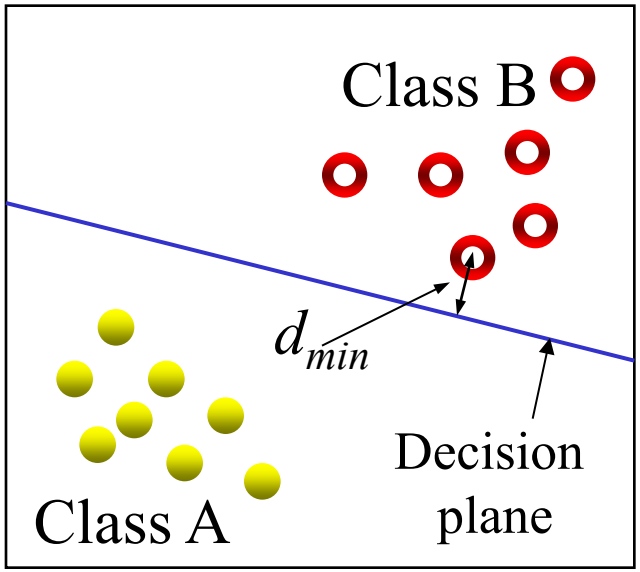
difficult

The difficulty of training process depends on the distance between data groups of each class (the more they depart the more easy, the more they close, the more difficult to train)

Classification Performance : How good the network has learnt



good



bad

performance of neural network in classify class can see from the minimum distance from input pattern to decision plane

Conclusions: single layer percentron networks

1. Function for each unit is in a form of function of input weighted sum

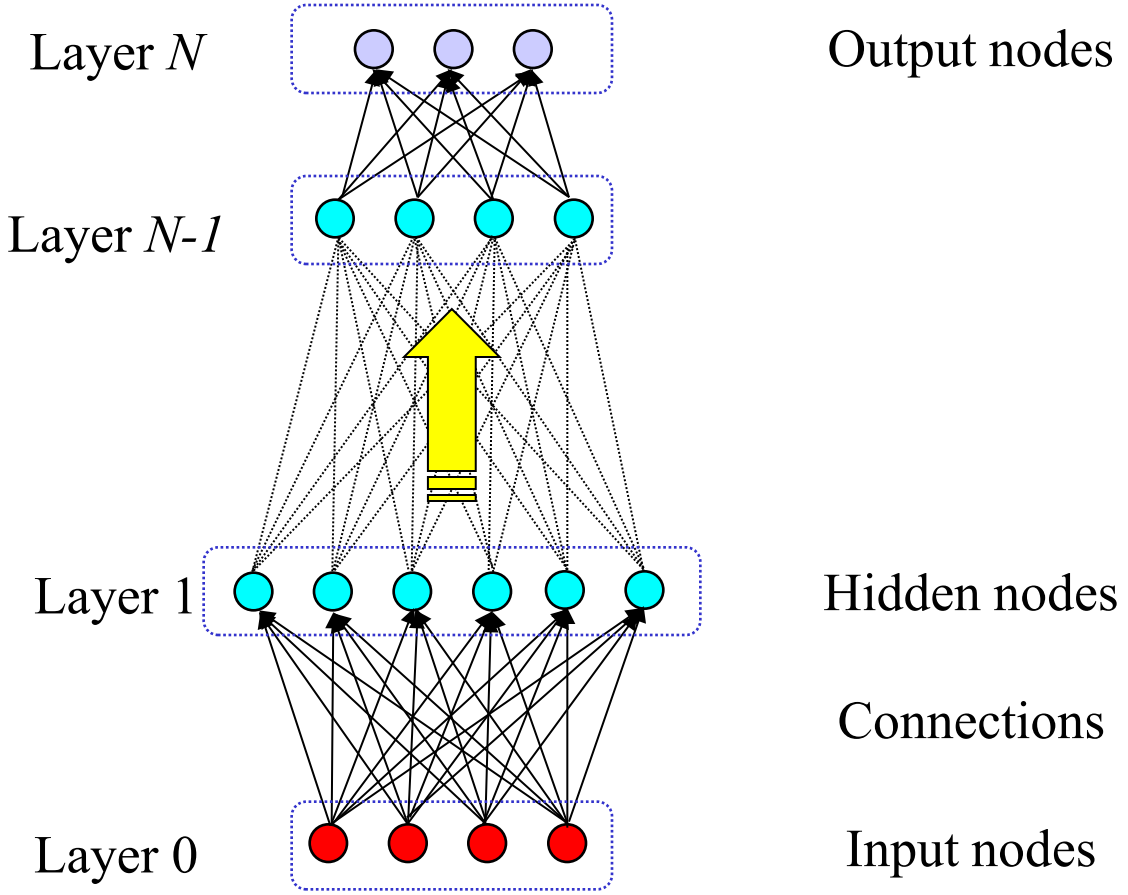
$$y_i = g(\sum_{j=1}^{N} w_{ij} x_j - \theta_i)$$

2. In the case of classifying data, Decision function will be in a form of Hyperplane
3. Limitation of a single layer network is data have to be linearly separable data
4. Training a neural net is done by adjusting the weight by the way of gradient descent method for decreasing the error as low as possible
5. Gradient descent method has a weakness in highly possible get stuck in local minima of error surface

Chapter 3

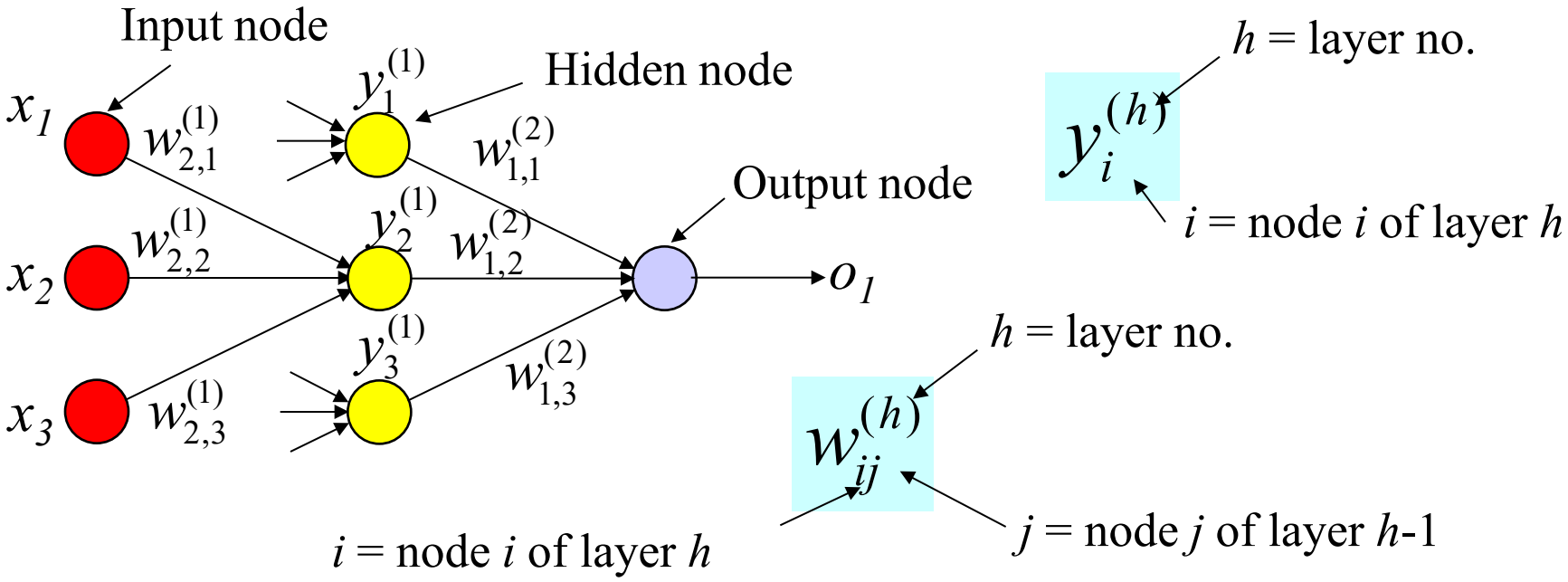
Multilayer Feedforward Networks

Multilayer Feedforward Network Structure



N-layer network

Multilayer Feedforward Network Structure (cont.)



Output for each node

$$y_i^{(h)} = f(w_{i,1}^{(h)} y_1^{(h-1)} + w_{i,2}^{(h)} y_2^{(h-1)} + w_{i,3}^{(h)} y_3^{(h-1)} + \cdots w_{i,m}^{(h)} y_m^{(h-1)} + \theta_i^{(h)})$$

$$= f(\sum_j w_{i,j}^{(h)} y_j^{(h-1)} + \theta_i^{(h)})$$

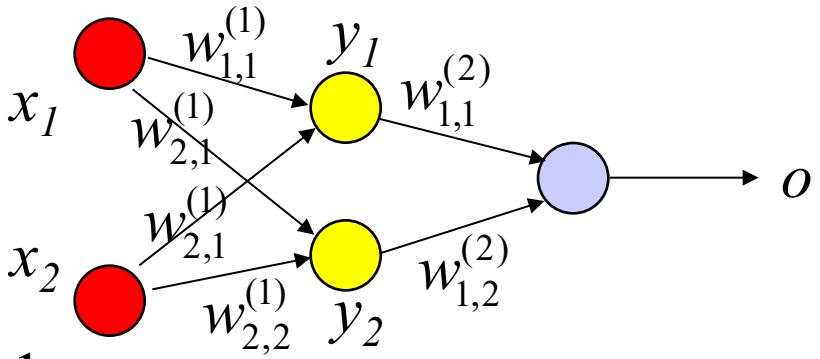
In which

$$y_j^{(0)} = x_j = \text{input } j \quad \text{and} \quad y_i^{(N)} = o_i = \text{Output } i$$

Multilayer Perceptron : How it works

Function XOR

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0



Layer 1

$$y_1 = f(w_{1,1}^{(1)} x_1 + w_{1,2}^{(1)} x_2 + \theta_1^{(1)})$$

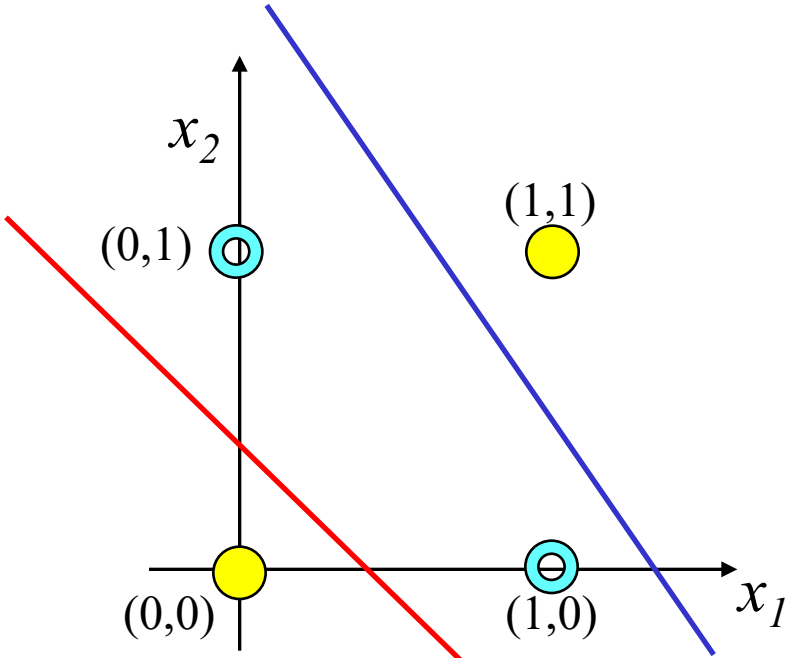
$$y_2 = f(w_{2,1}^{(1)} x_1 + w_{2,2}^{(1)} x_2 + \theta_2^{(1)})$$

Layer 2

$$o = f(w_{1,1}^{(2)} y_1 + w_{1,2}^{(2)} y_2 + \theta_1^{(2)})$$

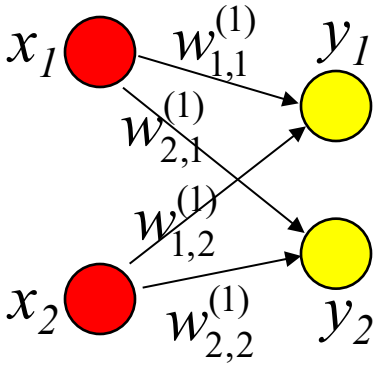
$f(\ )$ = step function

Multilayer Perceptron : How it works (cont.)



Outputs at layer 1

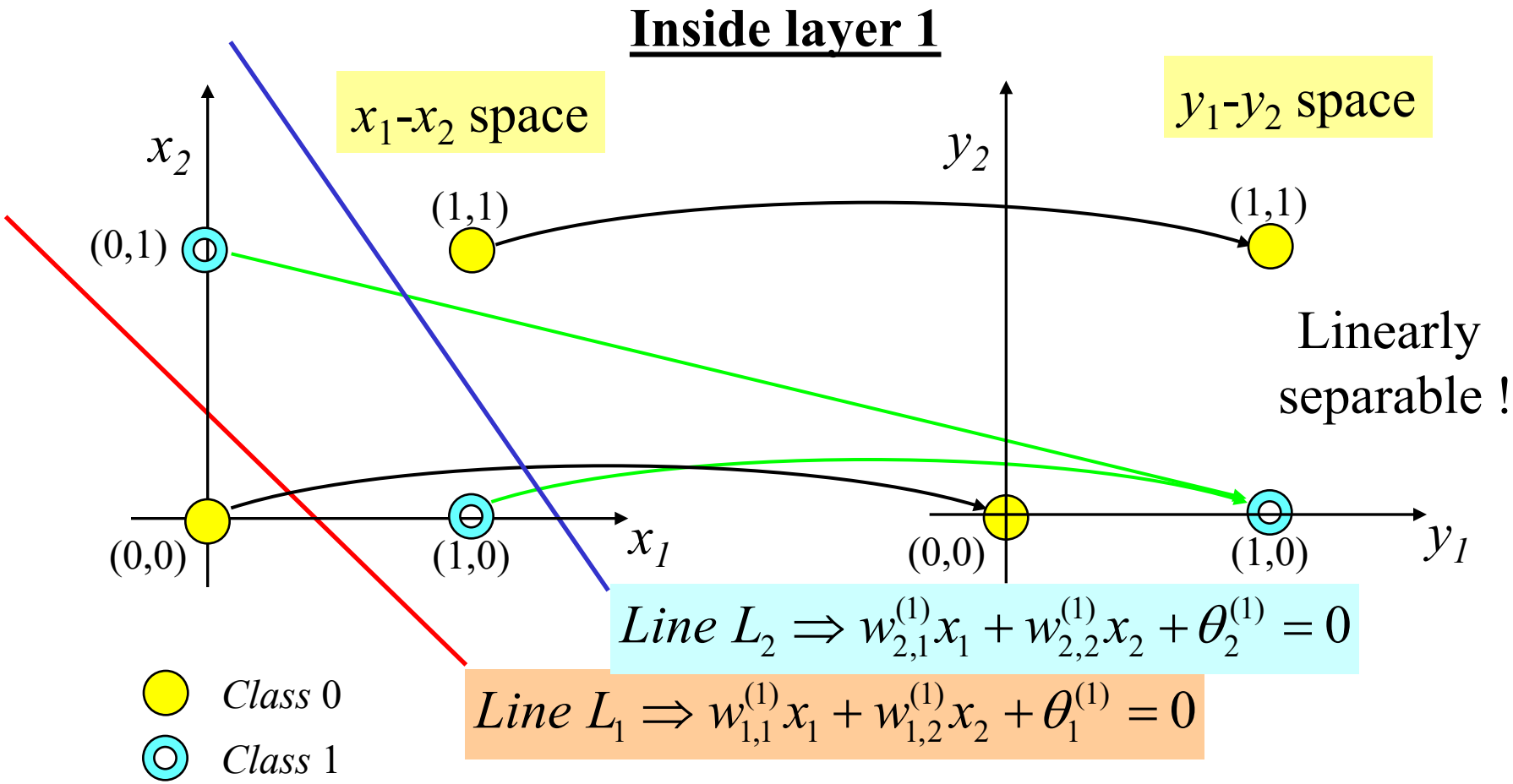
x_1	x_2	y_1	y_2
0	0	0	0
0	1	1	0
1	0	1	0
1	1	1	1



$$Line\ L_2 \Rightarrow w_{2,1}^{(1)} x_1 + w_{2,2}^{(1)} x_2 + \theta_2^{(1)} = 0$$

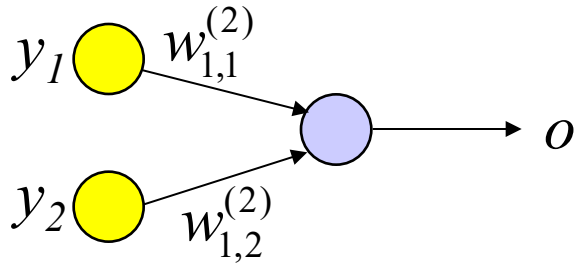
$$Line\ L_1 \Rightarrow w_{1,1}^{(1)} x_1 + w_{1,2}^{(1)} x_2 + \theta_1^{(1)} = 0$$

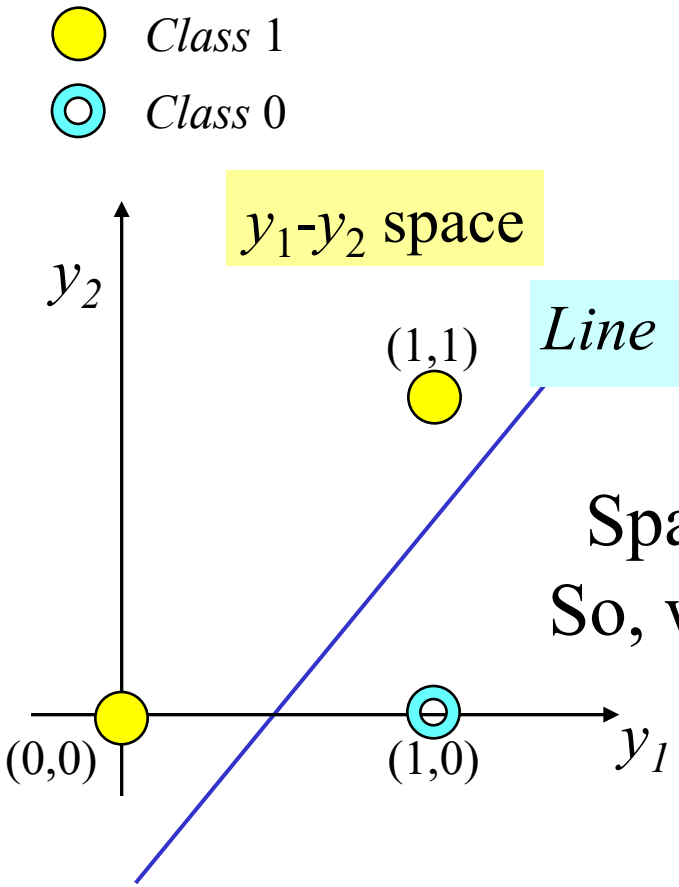
Multilayer Perceptron : How it works (cont.)



Multilayer Perceptron : How it works (cont.)

Inside output layer





$$Line \quad L_3 \Rightarrow w_{1,1}^{(2)} y_1 + w_{1,2}^{(2)} y_2 + \theta_1^{(2)} = 0$$

Space y_1-y_2 is linearly separable
So, we can use line L_3 separate
class 0 and class 1

hidden layers

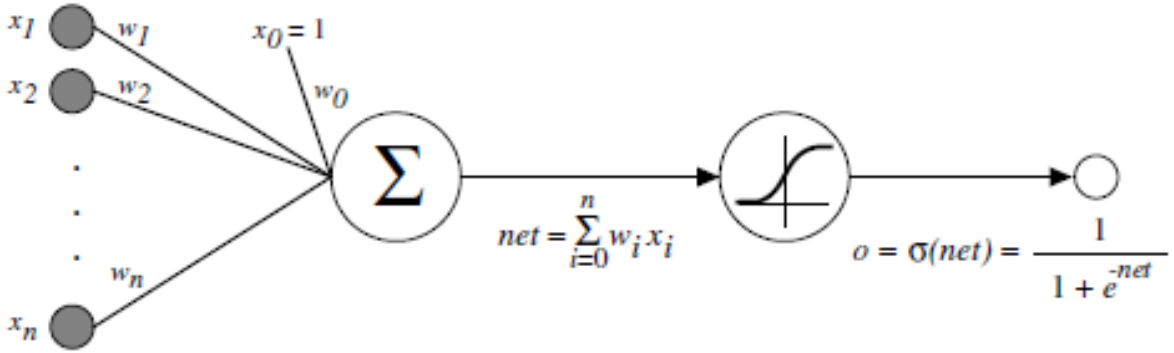
- try to converse incoming data in that layer to be linearly separable before send to the output layer

- at the final step at Hidden layer, data have to be linearly separable

- maybe we need hidden layer more than 1 layer to be able to converse into the form of linearly separable

- each Activation function for each layer don't need to be Thresholding function and the same function

Sigmoid Unit



$\sigma(x)$ is the sigmoid function

$$\frac{1}{1 + e^{-x}}$$

Nice property: $\frac{d\sigma(x)}{dx} = \sigma(x)(1 - \sigma(x))$

We can derive gradient decent rules to train

- One sigmoid unit
- *Multilayer networks* of sigmoid units $\rightarrow$ Backpropagation

Error Gradient for a Sigmoid Unit

$$
\begin{aligned}
\frac{\partial E}{\partial w_i} &= \frac{\partial}{\partial w_i} \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2 \\
&= \frac{1}{2} \sum_d \frac{\partial}{\partial w_i} (t_d - o_d)^2 \\
&= \frac{1}{2} \sum_d 2(t_d - o_d) \frac{\partial}{\partial w_i} (t_d - o_d) \\
&= \sum_d (t_d - o_d) \left(-\frac{\partial o_d}{\partial w_i} \right) \\
&= -\sum_d (t_d - o_d) \frac{\partial o_d}{\partial net_d} \frac{\partial net_d}{\partial w_i}
\end{aligned}
$$

But we know:

$$
\frac{\partial o_d}{\partial net_d} = \frac{\partial \sigma(net_d)}{\partial net_d} = o_d(1 - o_d)
$$

$$
\frac{\partial net_d}{\partial w_i} = \frac{\partial (\vec{w} \cdot \vec{x}_d)}{\partial w_i} = x_{i,d}
$$

So:

$$
\frac{\partial E}{\partial w_i} = -\sum_{d \in D} (t_d - o_d) o_d (1 - o_d) x_{i,d}
$$

Backpropagation Algorithm

Initialize all weights to small random numbers.
Until satisfied, Do

- For each training example, Do
  1. Input the training example to the network and compute the network outputs
  2. For each output unit k

$$\delta_k \leftarrow o_k(1 - o_k)(t_k - o_k)$$

  3. For each hidden unit h

$$\delta_h \leftarrow o_h(1 - o_h) \sum_{k \in outputs} w_{h,k}\delta_k$$

  4. Update each network weight $w_{i,j}$

$$w_{i,j} \leftarrow w_{i,j} + \Delta w_{i,j}$$

  where

$$\Delta w_{i,j} = \eta \delta_j x_{i,j}$$

Backpropagation Algorithm

- **For each $< x, t >$ in training sets, do**
  - **Propagate the input forward through the network:**
    1. Input the instance $\boldsymbol{x}$ to the network and compute the output o_u of every unit $\boldsymbol{u}$ in the network
  - **Propagate the errors backward through the network:**
    2. For each network output unit $\boldsymbol{k}$, calculate its error term δ_k

$$\delta_k \leftarrow o_k (1 - o_k)(t_k - o_k)$$

    3. For each hidden unit $\boldsymbol{h}$, calculate its error term δ_h

$$\delta_h \leftarrow o_h (1 - o_h) \sum_{k \in Outputs} \omega_{kh} \delta_k$$

    4. Update each network weight ω_{ji}

$$\omega_{ji} \leftarrow \omega_{ji} + \Delta\omega_{ji}(n)$$

Where

$$\Delta\omega_{ji}(n) = \beta \delta_j x_{ji} + \alpha\Delta\omega_{ji}(n-1)$$

Weight from unit i to unit j

Input from unit i to unit j

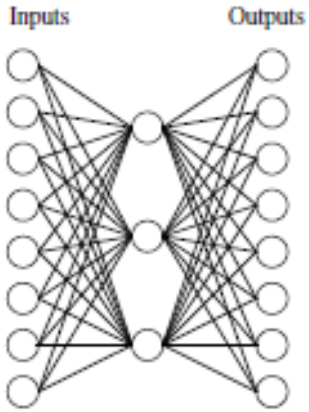
Momentum

Learning Rate

- Gradient descent over entire *network* weight vector
- Easily generalized to arbitrary directed graphs
- Will find a local, not necessarily global error minimum
  - In practice, often works well (can run multiple times)
- Often include weight *momentum* α

$$\Delta w_{i,j}(n) = \eta \delta_j x_{i,j} + \alpha \Delta w_{i,j}(n-1)$$

- Minimizes error over *training* examples
  - Will it generalize well to subsequent examples?
- Training can take thousands of iterations $\rightarrow$ slow!
- Using network after training is very fast

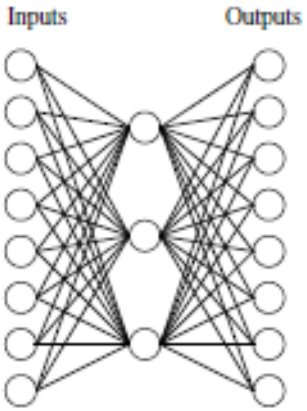


A target function:

Input		Output
10000000	$\rightarrow$	10000000
01000000	$\rightarrow$	01000000
00100000	$\rightarrow$	00100000
00010000	$\rightarrow$	00010000
00001000	$\rightarrow$	00001000
00000100	$\rightarrow$	00000100
00000010	$\rightarrow$	00000010
00000001	$\rightarrow$	00000001

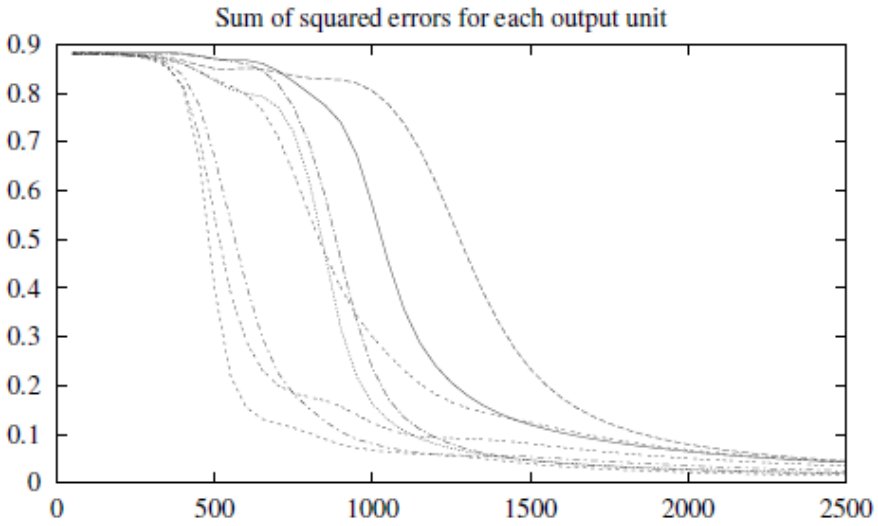
Can this be learned??

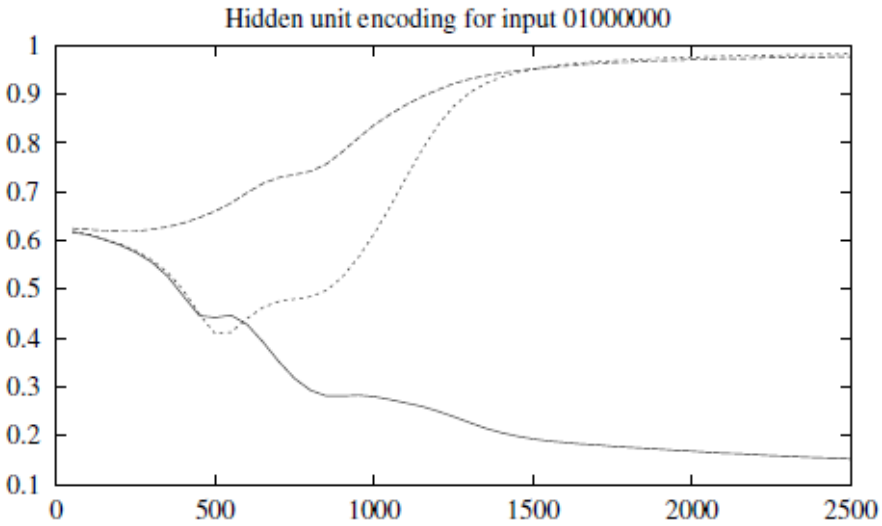
A network:

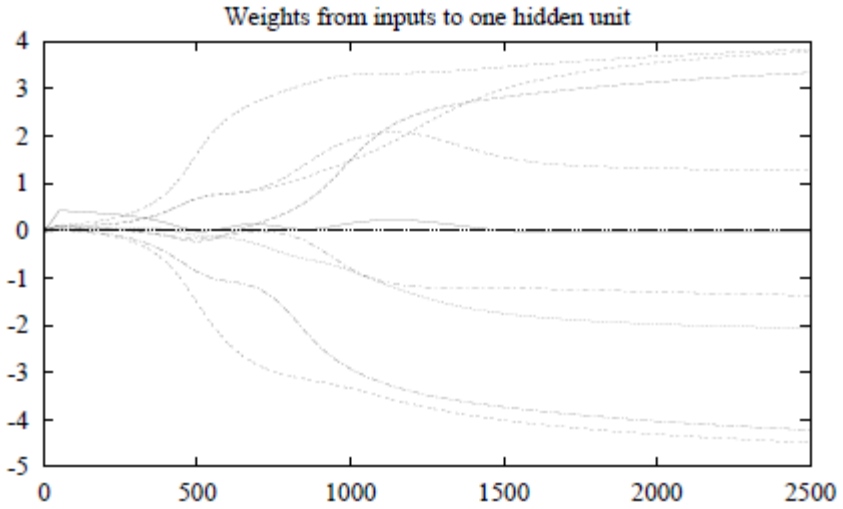


Learned hidden layer representation:

Input		Hidden Values				Output
10000000	→	.89	.04	.08	→	10000000
01000000	→	.01	.11	.88	→	01000000
00100000	→	.01	.97	.27	→	00100000
00010000	→	.99	.97	.71	→	00010000
00001000	→	.03	.05	.02	→	00001000
00000100	→	.22	.99	.99	→	00000100
00000010	→	.80	.01	.98	→	00000010
00000001	→	.60	.94	.01	→	00000001


Sum of squared errors for each output unit
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
0
500
1000
1500
2000
2500


Hidden unit encoding for input 01000000
1
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
500
1000
1500
2000
2500


Weights from inputs to one hidden unit
4
3
2
1
0
-1
-2
-3
-4
-5
0
500
1000
1500
2000
2500

Convergence of Backpropagation

Gradient descent to some local minimum

- Perhaps not global minimum...
- Add momentum
- Stochastic gradient descent
- Train multiple nets with different inital weights

Nature of convergence

- Initialize weights near zero
- Therefore, initial networks near-linear
- Increasingly non-linear functions possible as training progresses

Expressive Capabilities of ANNs

Boolean functions:

- Every boolean function can be represented by network with single hidden layer
- but might require exponential (in number of inputs) hidden units

Continuous functions:

- Every bounded continuous function can be approximated with arbitrarily small error, by network with one hidden layer [Cybenko 1989; Hornik et al. 1989]
- Any function can be approximated to arbitrary accuracy by a network with two hidden layers [Cybenko 1988].

ALVINN drives 70 mph on highways



